

Morgan SANTALUCIA

SICOM 2A
Année universitaire 2025/2026

GIPSA-lab
11 rue des Mathématiques 38402 Saint-Martin-d'Hères

Surveillance acoustique de synthèses mécanochimiques

du 18/05/2026 au 31/07/2026

Sous la supervision de :

- **Maître de stage : Pierre GRANJON – pierre.granjon@grenoble-inp.fr**
- **Tuteur école : Jerome MARS – jerome.mars@gipsa-lab.grenoble-inp.fr**

Confidentialité : non

**Ecole nationale
supérieure de physique,
électronique, matériaux**

Phelma
Bât. Grenoble INP - Minatec
3 Parvis Louis Néel - CS 50257
F-38016 Grenoble Cedex 01

Tél +33 (0)4 56 52 91 00
Fax +33 (0)4 56 52 91 03

<http://phelma.grenoble-inp.fr>

Remerciements

Je souhaiterais commencer ce travail de rédaction en adressant mes sincères remerciements à Monsieur Pierre Granjon, mon tuteur de stage et enseignant - chercheur à Grenoble INP, pour les missions confiées et la confiance accordée durant ces 11 semaines de stage, me permettant de faire l'expérience du monde des laboratoires de recherche et l'acquisition de compétences nouvelles. Je souhaiterais également remercier messieurs César Leroy et Assane Goumble ainsi que madame Chrystal Lopes, mes interlocuteurs à l'ICGM durant cette période de stage dont les travaux portent sur la mécanochimie, pour la bienveillance et l'implication tout au long de nos échanges, pour le suivi appliqué du travail que j'ai produit et pour la pertinence des retours qu'ils ont effectués. Je garde de ce stage de M1 une excellente vision du domaine de la recherche et de la collaboration entre les laboratoires.

Résumé

Le suivi de synthèses de mécanochimie par broyage à billes se heurte à l’opacité du réacteur : contrairement à une réaction en solution, aucune observation visuelle directe n’est possible pendant le broyage. Des travaux antérieurs [1] puis menés conjointement par le GIPSA-lab et l’Institut Charles Gerhardt Montpellier (ICGM) ont montré que l’analyse du signal acoustique mesurable lors de synthèses mécanochimiques permet d’accéder à des informations sur les phénomènes se déroulant à l’intérieur du réacteur, de façon non intrusive et *operando*. Ces travaux exploratoires étaient menés sous forme de scripts hétérogènes non réunis en un outil unique, et non optimisé pour l’utilisation par une équipe étrangère à Python.

Ce stage vise à consolider ces méthodes de traitement du signal et à les intégrer au sein d’une application unique, directement exploitable par l’équipe de mécanochimistes de l’ICGM sans connaissance préalable en programmation. L’application développée, sous Streamlit, propose une pipeline complète : prétraitement du signal (sous-échantillonnage, filtrage direct ou par la méthode IFIR selon la finesse de bande recherchée), calcul d’indicateurs statistiques par fenêtre glissante (kurtosis, négentropie), analyse énergétique par bandes de fréquences harmoniques de la fréquence de broyage, mise en parallèle avec des données de température, et export traçable des résultats. Une architecture logicielle centrée sur le mécanisme de `session_state` de Streamlit, ainsi que des registres centralisés, permettent de maintenir l’application accessible et évolutive.

Le développement suit un rythme hebdomadaire, alimenté par des retours réguliers de l’équipe de l’ICGM, aboutissant à une version stable (V7) livrée en fin de stage et adaptée à l’usage quotidien d’un utilisateur final. Un exemple d’exploitation autonome, portant sur la détection d’un intermédiaire réactionnel par corrélation entre indicateurs acoustiques et spectroscopie Raman, illustre la pertinence de l’outil au-delà d’un simple usage de démonstration.

Abstract

Monitoring mechanochemical ball-milling syntheses is inherently difficult : unlike solution-based reactions, the closed and opaque reactor prevents any direct visual observation during milling. Previous work carried out jointly by GIPSA-lab and the Institut Charles Gerhardt Montpellier (ICGM) showed that analysing the acoustic signal produced by the mill gives access to information about the phenomena occurring inside the reactor, in a non-intrusive and *operando* manner. These exploratory efforts, however, remained scattered across heterogeneous scripts with no unified pipeline, restricting their use to users familiar with Python.

This internship aims to consolidate these signal processing methods and integrate them into a single application, directly usable by the ICGM chemistry team without prior programming knowledge. The resulting Streamlit-based application provides a complete pipeline : signal preprocessing (downsampling, direct or IFIR filtering depending on the required transition bandwidth), statistical indicators computed over sliding windows (kurtosis, negentropy), energy analysis over frequency bands harmonic to the milling frequency, side-by-side visualization with temperature data, and traceable export of results. A software architecture built around Streamlit’s `session_state` mechanism, together with centralized registries, keeps the application maintainable and easy to extend.

Development followed a weekly release cycle, shaped by regular feedback from the ICGM team, resulting in a stable version (V7) delivered at the end of the internship and adopted in the daily workflow of an end user. A case study on detecting a reaction intermediate through correlation between acoustic indicators and Raman spectroscopy illustrates the relevance of the tool beyond a simple demonstration use case.

Glossaire

Acronyme	Définition
FIR	Finite Impulse Response (réponse impulsionnelle finie)
IFIR	Interpolated FIR (FIR interpolé, méthode de conception économique en ordre de filtre)
STFT	Short-Time Fourier Transform (transformée de Fourier à court terme)
PSD	Power Spectral Density (densité spectrale de puissance)
GIPSA-lab	Grenoble Images Parole Signal Automatique
ICGM	Institut Charles Gerhardt Montpellier
CNRS	Centre National de la Recherche Scientifique
SICOM	Signal, Image, Communication, Multimédia (filière de formation)
DDRS	Développement Durable et Responsabilité Sociétale

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Glossaire	iv
Liste des figures	ix
Liste des tableaux	x
Introduction	1
1 Lieu du stage, encadrement et suivi	4
1.1 GIPSA-lab	4
1.2 ICGM Montpellier	4
2 Contexte scientifique et technique	5
2.1 Mécanochimie et surveillance acoustique	5
2.2 Dispositif expérimental	5
2.3 Verrous du projet	7
2.4 Choix technologique : Streamlit	7
3 Méthodologie et conduite de projet	9
3.1 Organisation et temporalité	9
3.2 Architecture logicielle de l'application	11
3.2.1 Réglages transverses partagés	13
3.3 Documentation comme livrable	14

4	Développements techniques	16
4.1	Acquisition et prétraitement du signal	16
4.1.1	Lecture et ré-échantillonnage	16
4.1.2	Filtrage numérique : Remez direct vs IFIR	17
4.2	Indicateurs statistiques par fenêtre glissante	21
4.2.1	Kurtosis	22
4.2.2	Négentropie et synentropie	22
4.2.3	Facteur de forme et facteur de crête	23
4.2.4	Robustesse au changement de fenêtre de calcul	24
4.3	Méthode énergétique	24
4.4	Données complémentaires de température	27
4.5	Export et traçabilité	27
5	Résultats et validation sur cas réel	29
5.1	La pipeline de A à Z	29
5.1.1	Chargement et visualisation du signal brut	29
5.1.2	Sous-échantillonnage et filtrage passe-haut	30
5.1.3	Recadrage temporel du signal	30
5.1.4	Indicateurs statistiques : kurtosis et négentropie	30
5.1.5	Spectrogramme et bandes d'énergie harmoniques	31
5.1.6	Ajout de la température et mise en parallèle des indicateurs	32
5.2	Retours d'usage	33
5.3	Cas d'usage : mise en évidence d'un intermédiaire réactionnel	35
6	Enseignements et développement de compétences	37
6.1	Aspects techniques	37
6.2	Autonomie et conduite de projet	38
6.2.1	Usage de l'IA générative	39
	Conclusion et perspectives	41

Références bibliographiques	44
A Annexes	45
Annexes	45
Bilan de compétences et prévisionnel 3A	45
C2 – Concevoir une solution face à un problème technique	47

Table des figures

2.1	Dispositif expérimental : vue d'ensemble.	6
3.1	Diagramme de Gantt du déroulement du stage (18/05 - 31/07/2026), une colonne par semaine. Les repères (R1 à R9) renvoient aux réunions détaillées dans le tableau 3.2.	10
3.2	Architecture générale de l'application. App.py est le point d'entrée de l'application. Les pages du dossier pages/ sont indépendantes : chacune gère les choix de l'utilisateur et l'affichage qui lui est propre. Le traitement du signal est délégué aux fonctions du dossier utils/ , accessibles depuis toutes les pages via la façade FonctionsMS2026.py . Le dictionnaire partagé session_state , propre à Streamlit, permet de faire persister les données d'une page à l'autre sans recalcul.	13
4.1	Page Upload après chargement du fichier Son_SAM2.wav , correspondant à une synthèse de co-cristaux acide téréphtalique (TA) :1,4-diazabicyclo[2.2.2]octane (DABCO). Le fichier est échantillonné à 44 100 Hz pour 160 072 393 échantillons, soit une durée d'enregistrement de 3 629,76 s (environ 60 minutes).	16
4.2	Filtrage passe-haut du signal issu de la synthèse TA-DABCO ($f_{\text{stop}} = 10$ Hz, $f_{\text{pass}} = 40$ Hz, gain = 1). Le claquement de porte visible aux alentours de 151 s sur le signal ré-échantillonné est nettement atténué après filtrage. Ce type de filtrage sert plus généralement à isoler la bande de fréquences pertinente pour l'analyse.	17
4.3	Étapes de conception d'un filtre IFIR passe-haut ($F_s = 20\,000$ Hz, $f_c = 40$ Hz, $L = 8$). Le filtre modèle (a), conçu à fréquence réduite, est étiré pour atteindre F_s (b), ce qui introduit des répliques spectrales éliminées par le filtre de masquage (c) pour obtenir le filtre passe-bas complémentaire, ensuite inversé spectralement pour donner le filtre passe-haut final (d).	20
4.4	Validation du filtre IFIR sur un signal synthétique combinant une composante basse fréquence (5 Hz, sous f_{stop}), une composante haute fréquence (100 Hz, au-dessus de f_{pass}) et un bruit blanc additif. Le filtrage supprime la composante basse fréquence tout en préservant la composante haute fréquence.	20

4.5	Effet du <i>crop</i> sur les artefacts de bord de l'énergie normalisée par bande de fréquence, dans le cas où le signal contient encore quelques secondes enregistrées avant l'allumage du dispositif (ou après son arrêt). Sans <i>crop</i> (a), ce silence réel en début et fin d'enregistrement se traduit par une énergie anormalement faible à ces instants. Un léger <i>crop</i> du signal (b), pour exclure ces marges non pertinentes, suffit à retrouver une énergie normalisée cohérente sur l'ensemble de la plage temporelle.	26
5.1	Page Upload après chargement du signal brut de la synthèse TA-DABCO du 29/06/2026.	29
5.2	Signal sous-échantillonné (facteur 44) puis filtré (passe-haut, $f_{\text{stop}} = 10$ Hz, $f_{\text{pass}} = 40$ Hz, gain = 1) pour la synthèse TA-DABCO.	30
5.3	Kurtosis et négentropie calculés sur le signal préprocessé de la synthèse TA-DABCO.	31
5.4	Spectrogramme du signal préprocessé, synthèse TA-DABCO. Les harmoniques du signal apparaissent clairement sous forme de barres verticales aux multiples de la fréquence de broyage de 50 Hz	31
5.5	Énergie normalisée sur les bandes H_0 à H_3 (50, 100, 150 et 200 Hz, ± 10 Hz), synthèse TA-DABCO.	32
5.6	Mise en parallèle de la température, de l'énergie normalisée des bandes H_0 et H_1 , et de la négentropie, avec mise en évidence d'une zone temporelle d'intérêt, synthèse TA-DABCO.	33
5.7	Suivi d'une synthèse Phen-TFIB par broyage horizontal à 30 Hz (1 heure). De gauche à droite : température, signal Raman, énergie normalisée de la bande 140–160 Hz, kurtosis. Les zones A+B, C et D correspondent à trois régimes successifs identifiés de la réaction, la zone C étant associée à la présence d'un intermédiaire réactionnel.	36

Liste des tableaux

3.1	Historique des versions livrées durant le stage.	10
3.2	Réunions et jalons de suivi durant le stage.	11

Introduction

Contexte général

La mécano-chimie est une voie de synthèse dont les applications se multiplient depuis quelques années. Elle repose sur l'activation de réactions chimiques par une contrainte mécanique, ce qui en fait une alternative dite « verte » aux synthèses conventionnelles en solution, ces dernières reposant souvent sur des quantités importantes de solvants polluants. Parmi les procédés de mécano-chimie, le broyage à billes (*ball milling*) est l'un des plus répandus : un ou plusieurs récipients contiennent les réactifs solides ainsi que des billes qui, mises en mouvement, broient les composés.

Le suivi de l'évolution d'une synthèse par broyage à billes reste toutefois difficile. Contrairement à une réaction en solution, où la visualisation du milieu réactionnel renseigne le chimiste sur son avancement, le broyage à billes nécessiterait une possibilité de contrôle visuel mais les réactions se déroulent généralement dans des récipients en acier inoxydable, qui en privent totalement l'expérimentateur.

C'est pour lever ce verrou qu'une première étude [1] de l'Institut Charles Gerhardt Montpellier (ICGM) puis une seconde menée conjointement avec le laboratoire Grenoble Images Parole Signal Automatique (GIPSA-lab) [2] ont montré qu'une analyse du signal acoustique émis pendant le broyage permettait d'accéder à des informations sur les phénomènes se déroulant à l'intérieur du réacteur. Les changements observés dans le spectrogramme du signal enregistré ont pu être partiellement reliés à la nature du mouvement des billes, aux propriétés du mélange, ou encore à la présence de transformations chimiques. Cette possibilité de suivi *operando* (c'est-à-dire pendant que la réaction a lieu, sans interruption ni prélèvement) et non-invasif constitue une avancée notable pour l'étude des mécanismes réactionnels en mécano-chimie.

Un outil de visualisation

Des stages précédant celui-ci ont permis d'explorer plusieurs pistes de traitement d'enregistrements audios de réactions de synthèse mécano-chimique : extraction d'indicateurs statistiques sensibles aux événements impulsifs, calcul de spectrogrammes et d'indicateurs d'énergie par bande de fréquence. Ces travaux, menés dans un premier temps sous Matlab

puis portés en Python, ont permis de valider l'intérêt de plusieurs méthodes de traitement du signal pour ce cas d'usage.

Ces travaux exploratoires avaient vocation à tester des méthodes, dans l'optique de les intégrer à un outil unique, complet et directement utilisable par l'équipe de chimistes. Les scripts produits par les différents stagiaires coexistaient sous des formes hétérogènes, sans pipeline de traitement commune ni interface dédiée, ce qui en limitait la portée.

Objectifs du stage

Ce stage s'inscrit dans la continuité directe de ces travaux, avec un objectif de nature différente. Il s'agit de :

- reprendre les codes produits lors des stages précédents afin d'en comprendre les méthodes, d'en vérifier la validité et d'identifier les points perfectibles ;
- consolider et, quand cela est pertinent, améliorer ces méthodes, en y intégrant de nouvelles briques utiles à la robustesse ou à la performance de l'ensemble (par exemple, la conception de filtres IFIR (*Interpolated Finite Impulse Response*) pour réduire le coût de calcul du filtrage sur des bandes de transition étroites, (cf. chapitre 4)) ;
- intégrer l'ensemble de ces méthodes au sein d'une application unique, organisée autour d'une pipeline de traitement bien définie et reproductible, où chaque étape peut s'appuyer explicitement sur le résultat de la précédente ;
- rendre cet outil directement utilisable par l'équipe de mécanochimistes de l'ICGM, sans connaissance préalable en programmation, et le faire vivre par des cycles de retours réguliers avec ses utilisateurs finaux (cf. chapitre 3).

Le travail attendu consiste à transformer un ensemble de scripts prototypes en un produit fini, robuste et maintenable.

Plan du rapport

Le chapitre 1 présente très brièvement le laboratoire et l'équipe d'interlocuteurs. Le chapitre 2 détaille le contexte scientifique du projet (état de l'art, dispositif expérimental) ainsi que les verrous techniques ayant guidé les choix de conception. Le chapitre 3 décrit la méthodologie de conduite de projet adoptée, en particulier le positionnement par rapport aux travaux antérieurs et le cycle de collaboration mis en place avec l'équipe de Montpellier. Le chapitre 4 détaille les développements techniques réalisés : prétraitement du signal, indicateurs statistiques, méthode énergétique, données complémentaires de température,

export. Le chapitre 5 illustre l'ensemble de la pipeline sur un cas réel et en discute les résultats. Enfin, le chapitre 6 revient sur les enseignements techniques et humains tirés du stage avant la conclusion générale.

1. Lieu du stage, encadrement et suivi

1.1 GIPSA-lab

Le stage se déroule au laboratoire Grenoble Images Parole Signal Automatique (GIPSA-lab), spécialisé dans le traitement du signal et des images et l'automatique. Il s'effectue sous la supervision de Pierre Granjon, enseignant - chercheur à Grenoble INP, auteur des algorithmes de traitement du signal initialement développés sous Matlab dans le cadre des stages précédents, et qui constituent le point de départ des méthodes portées en Python. Le stage est réalisé au sein de l'équipe SAFE (*Safe, Controlled and Monitored Systems*), rattachée au Département Automatique et Sécurité des systèmes (DAS) du GIPSA-lab. Cette équipe travaille notamment sur le traitement du signal, la surveillance, le diagnostic et le pronostic des systèmes pour des problématiques de contrôle et de décision.

Dans le cadre de ce stage, l'expertise de l'équipe SAFE en traitement et en analyse des signaux est mobilisée pour développer l'application permettant d'exploiter les données issues des expériences de mécanochimie réalisées à l'ICGM.

1.2 ICGM Montpellier

Le projet associe également l'Institut Charles Gerhardt Montpellier (ICGM), au sein duquel une équipe de chimistes conduit les travaux expérimentaux de mécanochimie exploités par l'application développée durant ce stage.

Un déplacement au laboratoire de l'ICGM, décrit à la section 2.2, permet d'observer directement le dispositif expérimental à l'origine des signaux traités par l'application, de comprendre les enjeux et problématiques liés à la mécanochimie, de rencontrer physiquement les équipes intéressées par le produit du stage et d'établir un cahier des charges large pour les premières itérations de l'application.

2. Contexte scientifique et technique

2.1 Mécanochimie et surveillance acoustique

Le broyage à billes se prête mal aux techniques *operando* classiquement utilisées pour suivre une synthèse en solution. Des méthodes comme la spectroscopie Raman ou la mesure de température existent, mais elles nécessitent le plus souvent des adaptations matérielles du réacteur (fenêtre transparente, positionnement de la sonde...), ce qui limite leur mise en œuvre, en particulier sur des récipients classiques en acier inoxydable opaques aux rayonnements optiques.

L'analyse acoustique présente l'avantage de ne nécessiter aucune modification du dispositif expérimental : un microphone placé à proximité du réacteur suffit à enregistrer le signal sonore produit par les impacts et les mouvements des billes. Les études de référence de ce projet [2] et [1] ont mis en évidence plusieurs résultats motivant cette approche, notamment au niveau des variations du signal acoustique qui peuvent être associées à la formation d'intermédiaires réactionnels, un phénomène difficile à observer par d'autres moyens. Par ailleurs, les analyses statistique et spectrales du signal renseignent sur l'évolution du régime de broyage (nature des interactions entre billes : impacts localisés ou mouvement de roulement continu, dit *rolling*) et cette information est complémentaire de la spectroscopie Raman, qui renseigne sur la structure moléculaire, et de la mesure de température, qui reflète l'énergie dissipée pendant le broyage.

Le suivi acoustique constitue ainsi une approche non intrusive et directement transposable à des réacteurs opaques, ouvrant la voie à un suivi en temps réel des mécanismes réactionnels en mécanochimie. C'est ce constat qui a motivé, dans les stages précédant celui-ci, la mise en place d'un enregistrement du son pendant les synthèses ainsi que le développement des méthodes de traitement du signal.

2.2 Dispositif expérimental

Un déplacement au laboratoire de l'ICGM à Montpellier permet d'observer le dispositif expérimental sur lequel repose l'ensemble des données traitées par l'application (voir figure 2.1).

En préparation d'une manipulation, les composés chimiques dont on souhaite étudier la réaction sont placés dans un bol avec deux billes assurant le broyage. Une fois le bol installé sur le dispositif, la manipulation est lancée pour une durée prédéterminée, dans l'obscurité afin de ne pas perturber les prises de vue de la caméra thermique qui détecte des variations de température.

Durant une manipulation, plusieurs capteurs enregistrent simultanément :

- le **spectre Raman**, via une sonde dédiée ;
- le **son**, du réacteur d'une part et du broyeur d'autre part, via un microphone omnidirectionnel mais également un microphone hypercardioïde ;
- la **température**, via une caméra thermique ;
- la **vidéo**, via une caméra haute vitesse couplée à un éclairage LED, rendue possible par l'usage de récipients transparents en complément des récipients en acier inoxydable.

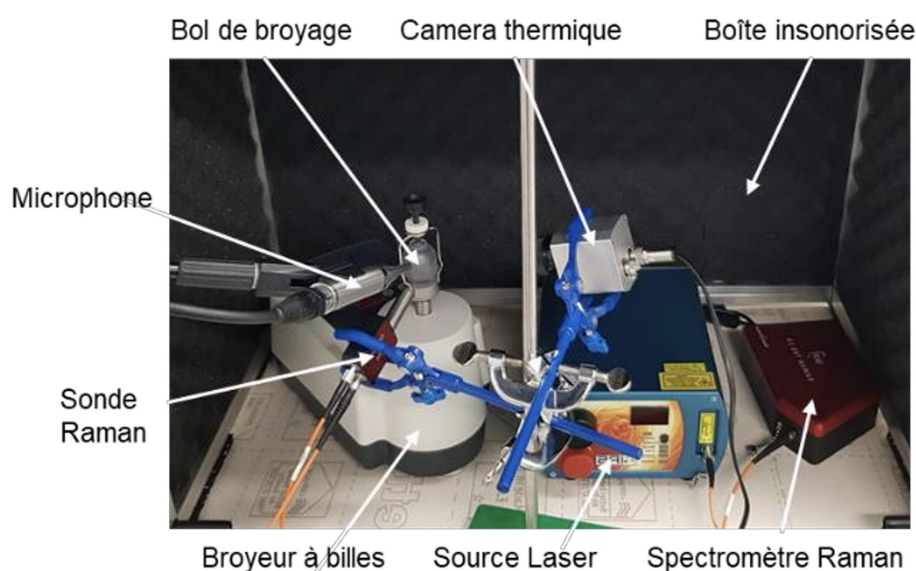


FIGURE 2.1 – Dispositif expérimental : vue d'ensemble.

Le signal Raman et la vidéo ne sont pas exploités par l'application développée durant ce stage ; ils correspondent à d'autres volets du dispositif expérimental global, non traités dans ce rapport. Le système d'acquisition auquel le microphone est relié produit en sortie un fichier `.wav`, qui constitue la donnée d'entrée de l'ensemble de la pipeline de traitement présentée au chapitre 4. La caméra thermique produit quant à elle un fichier de température au format `.dat`, exploité en tant que donnée complémentaire (cf. section 4, données de température).

2.3 Verrous du projet

Plusieurs verrous marquent le déroulement du stage.

Le premier est scientifique : la chimie reste une boîte noire, particulièrement pour un étudiant en signal, image, communication, multimédia (SICOM). Comprendre les phénomènes observés dans le réacteur ne doit pas être une barrière au traitement des signaux.

Le deuxième est organisationnel : le contact avec les utilisateurs finaux n'est possible qu'à distance, d'où la mise en place d'un cycle de retours hebdomadaires (cf. section 3.1).

Le troisième tient à l'objet d'étude lui-même : le réacteur étant fermé, il n'y a pas de vérité terrain sur ce qui s'y passe réellement. Les résultats ne peuvent être validés que sur des réacteurs adaptés à l'utilisation de la sonde Raman. De plus, le dispositif expérimental et de mesure évolue lui-même en cours de stage : le microphone sature sur les premiers enregistrements, jusqu'à ce que l'équipe de l'ICGM mène une étude sur le réglage du gain pour corriger le problème.

S'ajoute un verrou technique : les enregistrements représentent souvent une heure de signal à 44,1 kHz, soit des fichiers volumineux à traiter. Cela motive le sous-échantillonnage en amont de la pipeline (cf. chapitre 4). Le stage demande par ailleurs de mener de front le développement de l'application et la vérification de chaque étape de la pipeline de traitement.

2.4 Choix technologique : Streamlit

Le choix de Streamlit découle directement de ces contraintes. C'est une bibliothèque Python qui transforme un script de traitement en application web, sans développement front-end (HTML, CSS, JavaScript).

Elle s'intègre directement avec les méthodes déjà portées en Python (NumPy, SciPy), permet un prototypage rapide adapté au cycle de retours hebdomadaire (cf. section 3.1), et donne une interface simple (boutons, sliders, cases à cocher) utilisable sans connaissance de programmation. L'application tourne en local à partir des sources partagées sur GitHub, sans serveur dédié.

En contrepartie, l'interface est moins personnalisable qu'un développement web sur mesure, et certains calculs lourds (filtrage à bande étroite, spectrogramme sur signal long) demandent une attention particulière pour rester réactifs (cf. chapitre 4).

L'application vise également un déploiement sur Streamlit Community Cloud, la plateforme d'hébergement en ligne gratuite proposée par Streamlit pour partager une appli-

cation sans installation côté utilisateur. Les performances y sont nettement inférieures à un usage en local, notamment sur les calculs lourds (filtrage, spectrogramme). Ce déploiement ne vise donc pas un usage de production : il sert avant tout à faire découvrir l'application à la communauté et à inciter à l'utiliser en local.

3. Méthodologie et conduite de projet

3.1 Organisation et temporalité

Les deux premières semaines de stage sont consacrées à une prise de contact à Montpellier avec l'équipe de l'ICGM : découverte des acteurs du projet, des enjeux de la mécano-chimie et des produits des travaux des précédents stagiaires et du tuteur de stage. Un cahier des charges minimal, établi progressivement, aboutit à une version 0 (V0) de l'application, livrée sur le dépôt Git du groupe de travail au bout de deux semaines. Cette V0 se limite à intégrer, dans une interface commune et ergonomique, les quelques outils déjà codés et vérifiés.

Le développement suit alors un rythme hebdomadaire : une nouvelle version (voir table 3.1) publiée chaque vendredi sur GitHub, une semaine laissée aux utilisateurs pour tester et fournir des retours, puis une intégration de ces suggestions dans la version suivante. Le tuteur de stage assure en parallèle un suivi technique rapproché, proposant des pistes de méthodes à approfondir (cf. chapitre 4).

Version	Principaux apports
V0	Intégration des outils hérités dans une interface commune.
V1	Crop du signal après downsampling, export CSV décimé, normalisation du kurtosis indépendante de la fenêtre.
V2	Sélection d'une fondamentale et de ses harmoniques pour l'intégration d'énergie, export JPG, export automatique des paramètres de calcul.
V3	Crop déplacé dans la sidebar (partagé entre pages), correction d'un effet de bord sur les bandes d'énergie, ajout des indicateurs shape factor, crest factor et negentropy.
V4	Page de données de température (bêta), infobulles, documentation des indicateurs.
V5	Conception de filtres IFIR pour les hautes fréquences d'échantillonnage, optimisations de code, masquage du spectrogramme derrière une case à cocher pour la fluidité.

Version	Principaux apports
V6	Nettoyage général du code, automatisation des éléments redondants, rédaction du document INFOS.md.
V7	Réorganisation des pages, gestion de formats de fichiers de température variés, corrections de bugs.

TABLE 3.1 – Historique des versions livrées durant le stage.

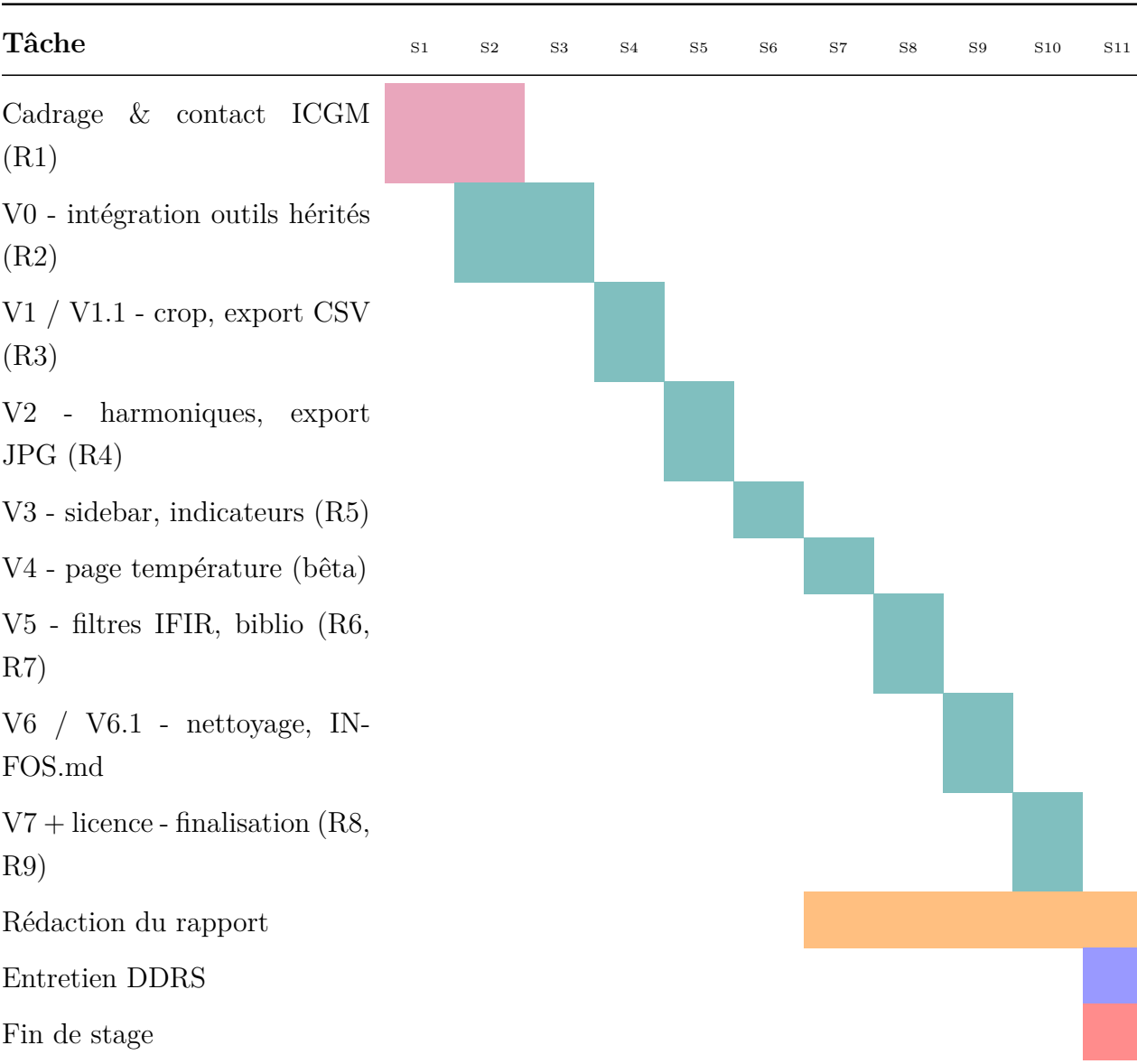


FIGURE 3.1 – Diagramme de Gantt du déroulement du stage (18/05 - 31/07/2026), une colonne par semaine. Les repères (R1 à R9) renvoient aux réunions détaillées dans le tableau 3.2.

TABLE 3.2 – Réunions et jalons de suivi durant le stage.

Rep.	Date	Objet
R1	19– 20/05/26	Déplacement à Montpellier (ICGM) - découverte du dispositif expérimental, prise de contact avec l'équipe, premier cahier des charges.
R2	05/06/26	Visio de présentation de la V0 avec le tuteur de stage et l'équipe de l'ICGM.
R3	11/06/26	Réunion de suivi avec le tuteur de stage sur la V1.
R4	17/06/26	Réunion de suivi avec le tuteur de stage sur la V2.
R5	25/06/26	Réunion de suivi avec le tuteur de stage sur la V3.
R6	01/07/26	Réunion de suivi avec le tuteur de stage sur la V4 - discussion sur les indicateurs statistiques et travail de bibliographie associé.
R7	09/07/26	Réunion de suivi avec le tuteur de stage sur la V5 - discussion sur la méthode de filtrage IFIR.
R8	21/07/26	Réunion de livraison du code au tuteur de stage (V7), avec explications et supports de prise en main.
R9	27/07/26	Présentation individuelle du travail final à l'interlocuteur privilégié de l'ICGM.
–	29/07/26	Entretien DDRS avec un représentant du groupe d'étude du GIPSA-lab sur la politique socio-environnementale du laboratoire.

Une partie du travail de factorisation de code (V5, V6) est réalisée avec l'appui d'un outil d'IA générative (Claude Sonnet 5), en gardant la maîtrise et la compréhension complète du code produit, point discuté ouvertement avec l'équipe lors du suivi hebdomadaire.

L'objectif fixé pour les dernières versions est de disposer d'une application propre, entièrement fonctionnelle et maintenable pour que l'équipe de l'ICGM puisse en faire une exploitation autonome.

3.2 Architecture logicielle de l'application

Streamlit repose sur un modèle d'exécution particulier, qu'il est utile de présenter avant de détailler l'architecture retenue : contrairement à une application web classique, où seule une portion de la page se met à jour lors d'une interaction, Streamlit ré-exécute l'intégralité du script Python correspondant à chaque interaction de l'utilisateur (déplacer un slider, cocher une case, changer de page). Ce modèle simplifie beaucoup l'écriture

du code, mais a une conséquence directe : sans mécanisme particulier, toute variable Python serait réinitialisée à chaque interaction, rendant impossible la moindre persistance d'information, y compris entre deux widgets d'une même page.

Streamlit répond à ce problème avec `st.session_state`, un dictionnaire persistant associé à la session de l'utilisateur, dont le contenu survit aux ré-exécutions et à la navigation d'une page à l'autre. C'est ce mécanisme qui structure directement l'architecture de l'application : plutôt que de transporter les données via des variables locales à chaque page, l'application centralise tout dans `session_state` : le signal chargé, les paramètres de prétraitement, les indicateurs déjà calculés, les figures prêtes à être exportées.

L'application s'organise ainsi en deux couches. Les six pages Streamlit (`Upload`, `Preprocessing`, `StatisticalIndicators`, `EnergyMethod`, `AdditionalData`, `Export`) ne contiennent que l'affichage et la récupération des choix de l'utilisateur : aucun calcul n'y est fait directement. Tout le traitement du signal est délégué à un dossier `utils/`, organisé en modules par thématique (lecture audio, filtrage, spectrogramme, bandes d'énergie, indicateurs statistiques, température, export...), accessibles depuis chaque page via une façade unique, `FonctionsMS2026.py`. La figure 3.2 illustre cette organisation : chaque page lit et écrit dans `session_state` pour communiquer avec les autres, sans jamais s'appeler directement entre elles.

Cette dépendance à `session_state` définit également l'ordre naturel de la pipeline : chaque page vérifie la présence des clés dont elle a besoin avant de s'exécuter. La page `StatisticalIndicators`, par exemple, vérifie que `original_data` est bien présent dans `session_state` et invite sinon l'utilisateur à revenir sur les pages précédentes. La pipeline n'est donc pas imposée par un mécanisme technique de Streamlit, mais par cette dépendance explicite aux clés attendues.

Le modèle de *rerun* pose deux problèmes supplémentaires, traités de façon uniforme dans l'application. D'une part, la valeur d'un widget (slider, case à cocher...) n'est conservée que tant qu'il reste affiché à l'écran : un widget qui disparaît de la page (changement de page, menu replié) perd sa valeur et se réinitialise par défaut à son prochain affichage. L'application contourne ce problème via une convention appliquée dans le module `widgets.py` : chaque widget persistant repose sur deux clés dans `session_state`, une clé `widget_X` propre à l'affichage courant et une clé `saved_X` qui conserve la dernière valeur choisie indépendamment de cet affichage. Les fonctions `fct_slider`, `fct_checkbox`, `fct_selectbox`, `fct_radio` et `fct_multiselect` encapsulent cette logique une fois pour toutes, et sont utilisées à la place des widgets natifs dans l'ensemble des pages.

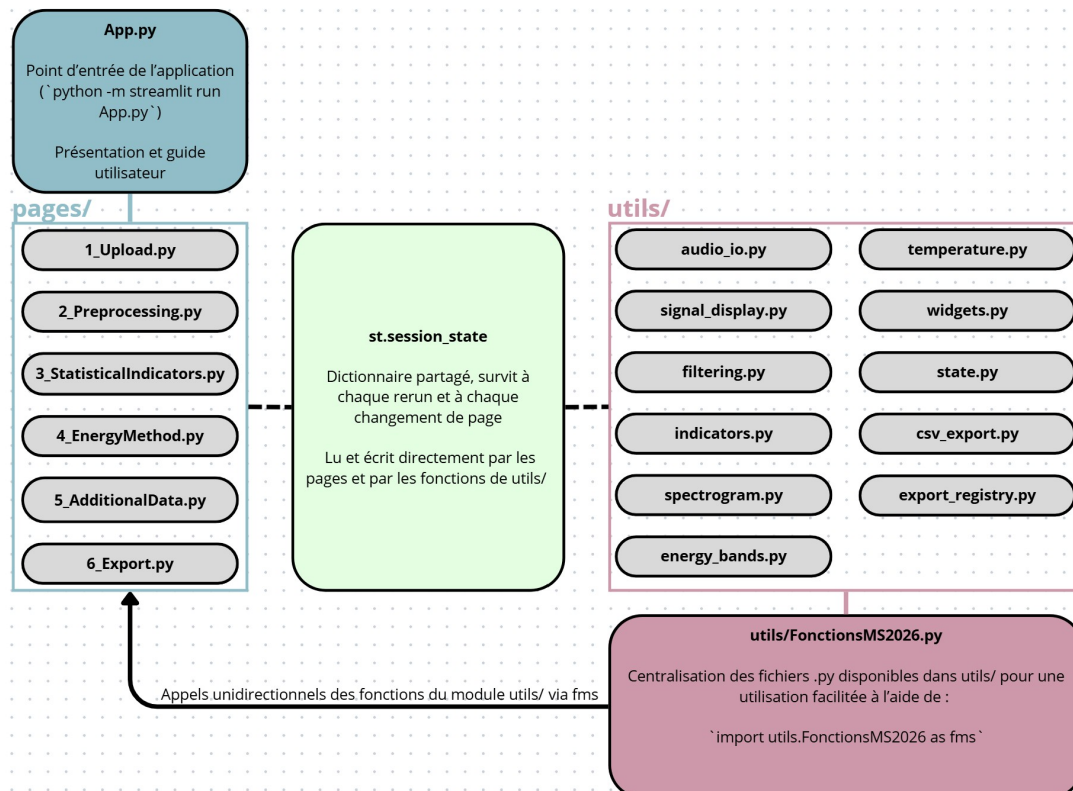


FIGURE 3.2 – Architecture générale de l’application. `App.py` est le point d’entrée de l’application. Les pages du dossier `pages/` sont indépendantes : chacune gère les choix de l’utilisateur et l’affichage qui lui est propre. Le traitement du signal est délégué aux fonctions du dossier `utils/`, accessibles depuis toutes les pages via la façade `FonctionsMS2026.py`. Le dictionnaire partagé `session_state`, propre à Streamlit, permet de faire persister les données d’une page à l’autre sans recalcul.

3.2.1 Réglages transverses partagés

Deux réglages transverses, le *crop* temporel du signal et le facteur de décimation d’affichage, sont soumis à la même contrainte de partage entre pages que les widgets décrits ci-dessus, mais s’appliquent à l’ensemble de l’application plutôt qu’à une seule page. Plutôt que de dupliquer leur logique dans chacune des six pages, l’application centralise leur rendu dans deux fonctions, `render_crop_sidebar` et `render_decimation_sidebar`, appelées en tête de chaque page qui en a besoin (cf. figure 3.2).

`render_crop_sidebar` affiche dans la barre latérale les bornes de temps à conserver, et recalcule à chaque appel le signal recadré correspondant (`resampled_data`) ainsi que sa version filtrée (`resampled_data_filtered`) à partir des versions complètes conservées en `session_state` (`resampled_data_full`, `resampled_data_filtered_full`) : le *crop* n’est donc jamais appliqué qu’une seule fois de façon définitive, mais recalculé à la demande, ce qui permet à l’utilisateur de revenir en arrière sur son réglage sans perdre le signal d’origine. Un *crop* invalide (borne de début supérieure ou égale à la borne de fin)

désactive silencieusement le recadrage plutôt que de bloquer l'application, et retombe sur le signal complet.

`render_decimation_sidebar` fonctionne sur le même principe pour le facteur de décimation d'affichage : une valeur unique, stockée dans `session_state["display_decimation"]`, est lue par toutes les figures temporelles de l'application (indicateurs statistiques, bandes d'énergie, température) pour réduire le nombre de points tracés à l'écran, sans jamais affecter les valeurs sous-jacentes ni les exports (cf. section 4.3 et section 4, export). Ce facteur est distinct de la décimation adaptative appliquée au spectrogramme lui-même (cf. section 4.3), qui répond à une contrainte différente : celle-ci vise à rendre affichable une matrice pouvant atteindre plusieurs millions de points, quel que soit le réglage choisi par l'utilisateur.

D'autre part, un calcul coûteux (ré-échantillonnage, filtrage, spectrogramme) pourrait en théorie être relancé à chaque interaction, même lorsque ses paramètres n'ont pas changé. L'application s'appuie ici sur le décorateur `st.cache_data`, apposé sur les fonctions de calcul les plus coûteuses : Streamlit retient le résultat associé à un jeu de paramètres donné et ne relance le calcul que si l'un d'eux change réellement. Ce mécanisme est distinct de `session_state` : il ne sert pas à transporter une donnée d'une page à l'autre, mais à éviter un recalcul inutile, ce qui est déterminant compte tenu de la volumétrie des signaux traités (cf. chapitre 2).

Deux registres centralisés complètent cette architecture. `INDICATOR_SPECS` liste chaque indicateur statistique (kurtosis, négentropie, shape factor, crest factor) avec ses paramètres par défaut, sa formule et son export associé : ajouter un nouvel indicateur ne demande de modifier qu'un seul endroit du code, plutôt que chaque page qui l'utilise. De la même manière, `export_registry` centralise la liste des figures et données exportables : la page **Export** construit ses cases à cocher directement depuis ce registre, sans connaître le détail de chaque type de graphe.

3.3 Documentation comme livrable

Le stage devant se terminer par la remise de l'application à l'équipe de l'ICGM, la documentation a été traitée comme un livrable à part entière, et non comme une étape finale. Composée de deux documents, elle se destine à deux publics différents.

Un **README** décrit l'application du point de vue de l'utilisateur : installation, usage page par page, explication en langage accessible des méthodes de traitement du signal employées (downsampling, filtrage, indicateurs, spectrogramme...). Un second document, `INFOS.md`, s'adresse à qui souhaiterait reprendre ou faire évoluer le code : il détaille l'architecture pré-

sentée ci-dessus et propose des recettes pas à pas pour les modifications les plus courantes (ajouter un indicateur, ajouter un élément exportable, ajouter une page).

Cette distinction découle des registres centralisés décrits en section 3.2 : la documentation développeur explique où intervenir pour chaque type de modification, ce qui limite le risque qu'une évolution future casse une partie du code non concernée par le changement.

4. Développements techniques

4.1 Acquisition et prétraitement du signal

4.1.1 Lecture et ré-échantillonnage

Le fichier `.wav` du signal acoustique (cf. section 2.2) constitue la donnée d'entrée de l'ensemble de la pipeline. Il est échantillonné à 44,1 kHz (4.1), fréquence standard pour l'enregistrement audio mais largement supérieure au besoin réel : l'essentiel de l'information acoustique liée au broyage se concentre à des fréquences plus basses (quelques centaines de Hz au maximum), le reste du spectre ne contenant que du bruit haute fréquence sans intérêt pour l'analyse.

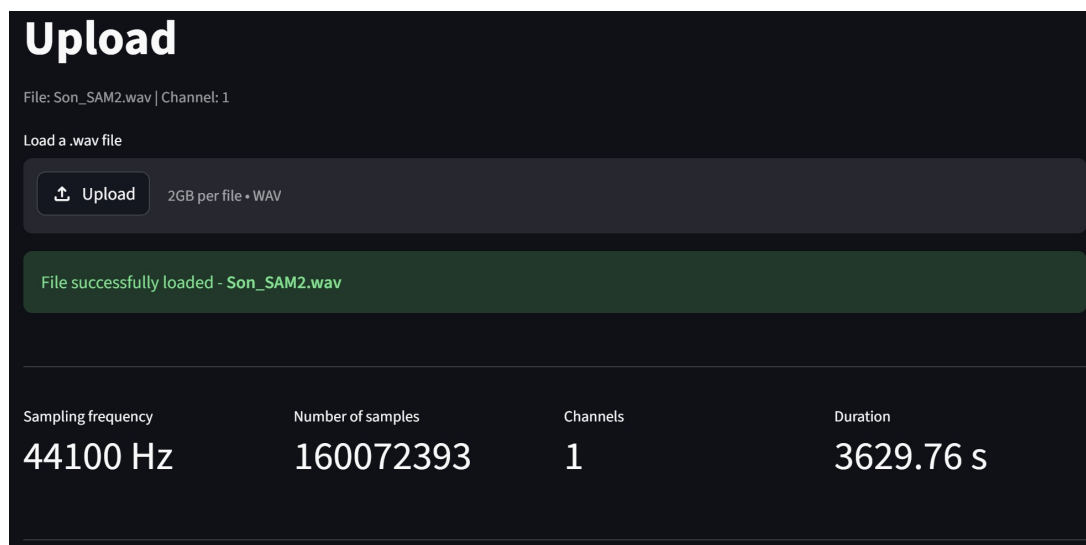


FIGURE 4.1 – Page Upload après chargement du fichier `Son_SAM2.wav`, correspondant à une synthèse de co-cristaux acide téréphtalique (TA) :1,4-diazabicyclo[2.2.2]octane (DABCO). Le fichier est échantillonné à 44 100 Hz pour 160 072 393 échantillons, soit une durée d'enregistrement de 3 629,76 s (environ 60 minutes).

L'application applique donc un sous-échantillonnage (*downsampling*) avant tout traitement, via la fonction `scipy.signal.resample_poly`. Cette étape réduit à la fois le volume de données à manipuler (cf. verrou identifié en section 2) et le temps de calcul des étapes suivantes, en particulier du filtrage (cf. section 4.1.2). Un facteur de sous-échantillonnage de 44 est appliqué par défaut, ramenant la fréquence d'échantillonnage aux alentours de 1 kHz : cette valeur, déterminée empiriquement offre une marge suffisante au-dessus des

fréquences d'intérêt tout en réduisant la volumétrie d'un facteur 44. L'utilisateur peut modifier ce facteur depuis l'interface si son cas d'usage l'exige.

Pour rester réactive malgré la taille des fichiers, l'application ne convertit en tableau exploitable que la portion du signal effectivement affichée à l'écran dans la page **Upload**. Uniquement la fenêtre temporelle visualisée, plutôt que l'intégralité du fichier. Ce n'est qu'à l'étape de prétraitement, une fois le canal sélectionné, que le signal est chargé dans son intégralité pour être sous-échantillonné.

4.1.2 Filtrage numérique : Remez direct vs IFIR

Une fois le signal sous-échantillonné, l'application permet d'appliquer un filtre numérique à réponse impulsionnelle finie (FIR) pour ne conserver que la bande de fréquences pertinente pour l'analyse. Trois types de filtres sont proposés : passe-haut, passe-bas et passe-bande. Dans le contexte des enregistrements de broyage, un filtre passe-haut est en pratique le plus utilisé, pour supprimer le bruit environnemental basse fréquence (par exemple une porte qui claque à proximité du dispositif comme sur la figure 4.2).

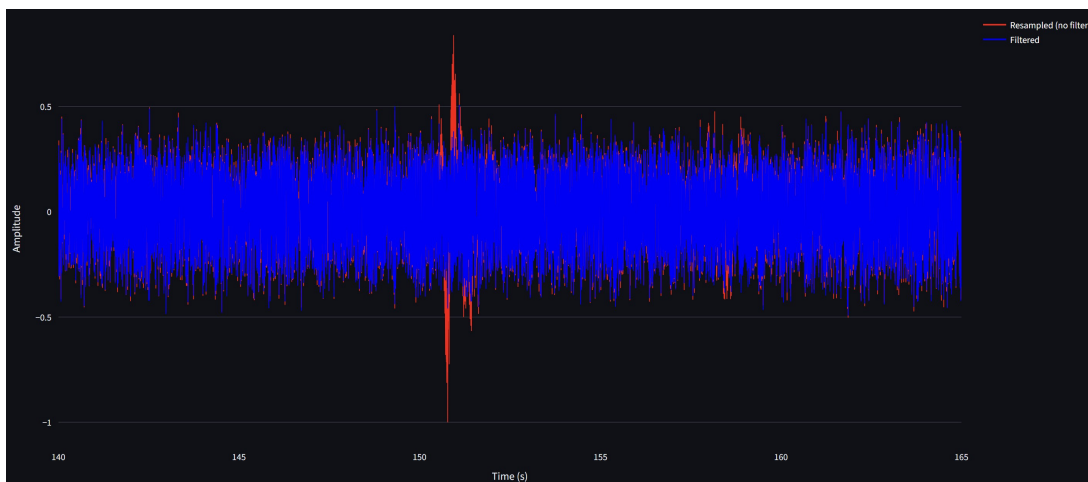


FIGURE 4.2 – Filtrage passe-haut du signal issu de la synthèse TA-DABCO ($f_{\text{stop}} = 10$ Hz, $f_{\text{pass}} = 40$ Hz, gain = 1). Le claquement de porte visible aux alentours de 151 s sur le signal ré-échantillonné est nettement atténué après filtrage. Ce type de filtrage sert plus généralement à isoler la bande de fréquences pertinente pour l'analyse.

Le filtre est par défaut conçu par la méthode de Remez, qui permet de déterminer les coefficients d'un filtre FIR à partir d'un gabarit défini par des fréquences de coupure de bande atténuée (*stopband*, f_{stop}) et de bande passante (*passband*, f_{pass}). Dans l'application, une valeur de $\delta_p = 1/1000$ est utilisée comme paramètre d'atténuation dans l'estimation de l'ordre du filtre. Cette valeur intervient dans le calcul préalable du nombre de coefficients nécessaire, avant la synthèse effective du filtre par l'algorithme de Remez.

Pour un filtre passe-bas de fréquence d'échantillonnage F_s , la largeur de la bande de transition est définie par la quantité normalisée

$$\Delta F = \frac{|f_{\text{pass}} - f_{\text{stop}}|}{F_s}. \quad (4.1)$$

Le code estime ensuite l'ordre nécessaire du filtre à partir d'une relation empirique dépendant de la largeur de cette bande de transition et du niveau d'atténuation recherché. Pour les filtres passe-haut et passe-bas, l'expression utilisée est de la forme

$$N \approx \frac{20 \log_{10} \left(\frac{1}{\pi \delta_p} \right) - 10 \log_{10} \left(\log_{10} \left(\frac{1}{\delta_p} \right) \right)}{10 \log_{10}(e) \log \left[\cot \left(\frac{\pi - 2\pi \Delta F}{4} \right) \right]} \quad (4.2)$$

où $\delta_p = 1/1000$ dans l'application. L'ordre obtenu est ensuite arrondi à l'entier supérieur, rendu impair afin d'obtenir un filtre FIR symétrique à phase linéaire, puis limité à une valeur maximale de 10001 coefficients.

Cette estimation permet notamment de mettre en évidence l'influence de la largeur de la bande de transition. Plus ΔF est petit, c'est-à-dire plus la bande de transition est étroite par rapport à la fréquence d'échantillonnage, plus l'ordre N requis est élevé. Ce comportement devient particulièrement contraignant lorsque l'on cherche à isoler une bande de fréquences étroite au sein d'un signal échantillonné à une fréquence beaucoup plus élevée : le nombre de coefficients peut alors devenir très important, ce qui augmente le coût de calcul de la convolution du filtre avec le signal.

Une fois l'ordre déterminé, les coefficients du filtre FIR sont effectivement calculés avec la fonction `scipy.signal.remez`. Pour un filtre passe-haut ou passe-bas, le gabarit est défini par les fréquences 0, f_{stop} , f_{pass} et la fréquence de Nyquist $F_s/2$. Pour un filtre passe-bande, deux bandes de transition sont définies autour de la bande conservée. Les coefficients obtenus sont ensuite éventuellement multipliés par le gain demandé par l'utilisateur.

Il est important de distinguer ici l'estimation de l'ordre et la synthèse du filtre : la formule précédente ne constitue pas à elle seule la méthode de Remez. Elle sert à déterminer une longueur de filtre adaptée, puis l'algorithme de Remez est utilisé pour calculer les coefficients optimisés correspondant au gabarit spécifié.

Pour les filtres passe-haut et passe-bas, l'application dispose également d'une méthode de conception alternative, dite IFIR (*Interpolated FIR*). Cette méthode est utilisée automatiquement lorsque la largeur de la bande de transition normalisée devient suffisamment faible, c'est-à-dire lorsque

$$\Delta F < 0,1. \quad (4.3)$$

Le choix de ce seuil constitue une règle de sélection implémentée dans l'application : il ne correspond pas à une démonstration générale selon laquelle l'IFIR serait systématiquement moins coûteux que le filtre direct pour toutes les configurations.

Le principe de l'IFIR consiste à décomposer le filtre en deux filtres de longueur plus faible :

- un filtre modèle, conçu à une fréquence d'échantillonnage réduite F_s/L (L entier, dit facteur d'étirement temporel), où la bande de transition représente une proportion plus importante de la fréquence d'échantillonnage : son ordre peut ainsi être fortement réduit ;
- un filtre de masquage, conçu à la fréquence d'échantillonnage complète F_s , qui élimine les répliques spectrales introduites par l'étirement du filtre modèle, réalisé en insérant $L - 1$ zéros entre chaque coefficient du filtre modèle.

Le facteur d'étirement L est choisi automatiquement par l'application. Plusieurs valeurs entières de L sont testées, jusqu'à une valeur maximale de 30. Pour chacune, l'ordre du filtre modèle (évalué à F_s/L) et celui du filtre de masquage (évalué à F_s) sont estimés à l'aide de la relation empirique de Herrmann, Rabiner et Chan [3] :

$$N \approx \frac{D_\infty(\delta_p, \delta_s)}{\Delta F} - f(\delta_p, \delta_s) \Delta F + 1 \quad (4.4)$$

où

$$D_\infty(\delta_p, \delta_s) = \left(0.005309 (\log_{10} \delta_p)^2 + 0.07114 \log_{10} \delta_p - 0.4761\right) \log_{10} \delta_s \\ - \left(0.00266 (\log_{10} \delta_p)^2 + 0.5941 \log_{10} \delta_p + 0.4278\right) \quad (4.5)$$

$$f(\delta_p, \delta_s) = 11.01217 + 0.51244 (\log_{10} \delta_p - \log_{10} \delta_s) \quad (4.6)$$

Dans l'implémentation de l'IFIR, une même valeur de $1/1000$ est utilisée pour les niveaux d'erreur associés aux bandes passante et atténuée, soit $\delta_p = \delta_s = 1/1000$; le second terme de $f(\delta_p, \delta_s)$ s'annule alors. La valeur de L retenue est celle qui minimise la somme des deux ordres ainsi estimés.

Le filtre modèle est d'abord conçu à la fréquence réduite F_s/L par la méthode de Remez. Ses coefficients sont ensuite étirés par insertion de $L - 1$ zéros entre deux coefficients successifs. Cette opération permet d'obtenir un filtre défini à la fréquence d'échantillonnage originale F_s , mais introduit plusieurs répliques spectrales de la réponse du filtre modèle. Le filtre de masquage est alors conçu à F_s afin de supprimer les répliques indésirables et de conserver la bande fréquentielle recherchée.

Pour un filtre passe-haut, l'IFIR est construit à partir d'un filtre passe-bas complémentaire. Après conception du filtre passe-bas IFIR, une inversion spectrale est appliquée afin d'obtenir le filtre passe-haut final. Cette opération revient à conserver la réponse complémentaire du filtre passe-bas tout en maintenant la symétrie des coefficients et la phase linéaire du filtre FIR.

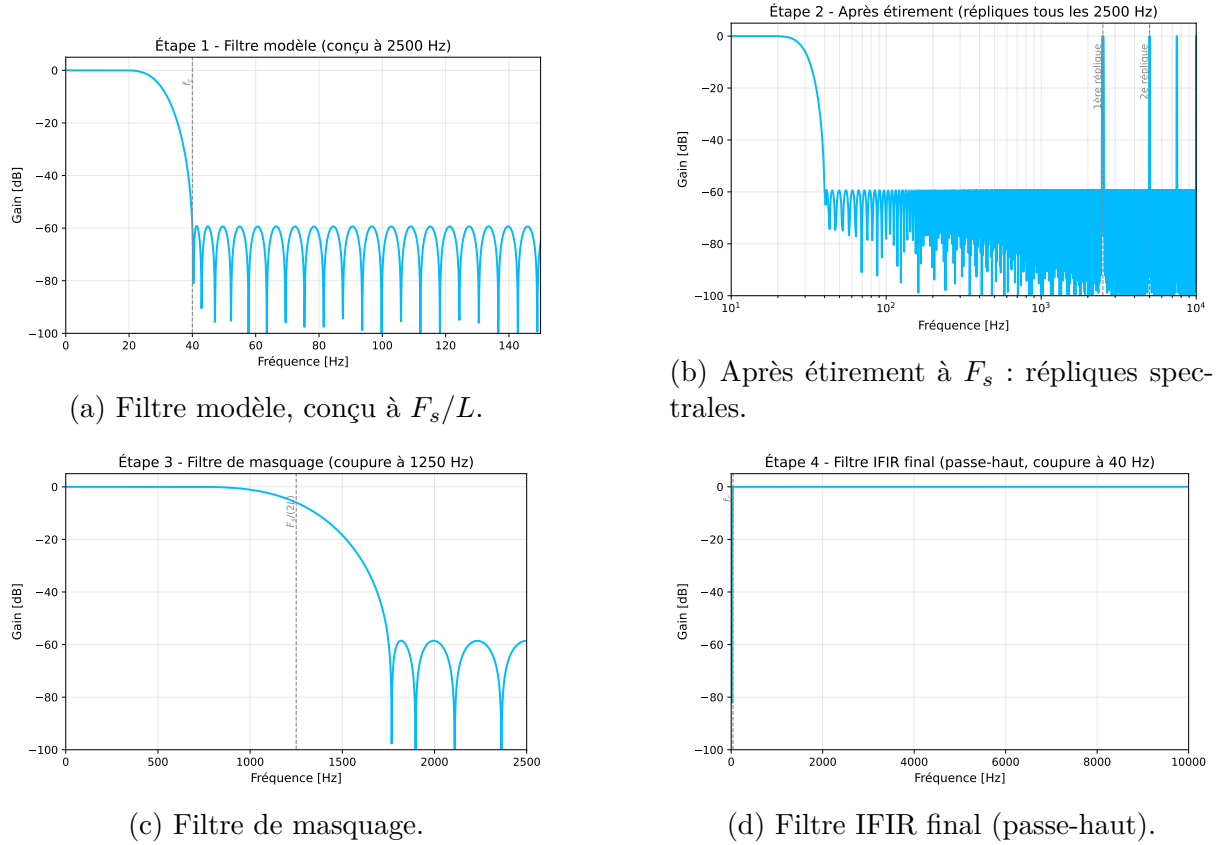


FIGURE 4.3 – Étapes de conception d'un filtre IFIR passe-haut ($F_s = 20\,000$ Hz, $f_c = 40$ Hz, $L = 8$). Le filtre modèle (a), conçu à fréquence réduite, est étiré pour atteindre F_s (b), ce qui introduit des répliques spectrales éliminées par le filtre de masquage (c) pour obtenir le filtre passe-bas complémentaire, ensuite inversé spectralement pour donner le filtre passe-haut final (d).

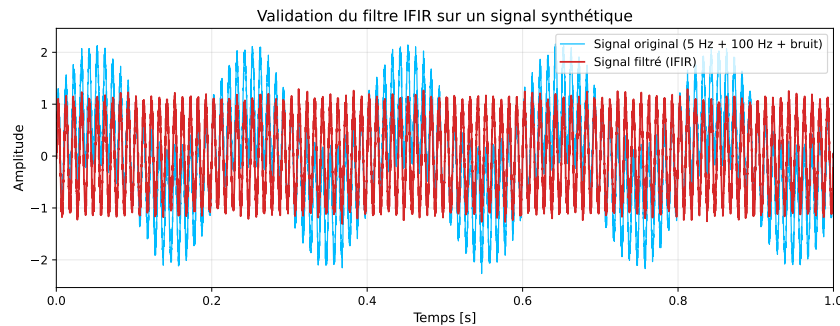


FIGURE 4.4 – Validation du filtre IFIR sur un signal synthétique combinant une composante basse fréquence (5 Hz, sous f_{stop}), une composante haute fréquence (100 Hz, au-dessus de f_{pass}) et un bruit blanc additif. Le filtrage supprime la composante basse fréquence tout en préservant la composante haute fréquence.

Une fois les coefficients du filtre obtenus, que ce soit par la conception directe avec Remez ou par la construction IFIR, le filtre est appliqué au signal par convolution. L'application utilise `scipy.signal.oaconvolve` [4], qui implémente une convolution de type *overlap-add* basée sur des transformées de Fourier rapides (FFT). Ce choix est particulièrement intéressant pour les filtres comportant un grand nombre de coefficients, pour lesquels une convolution directe devient rapidement coûteuse. Dans les essais réalisés sur des signaux échantillonnés à 44,1 kHz sans sous-échantillonnage préalable, l'utilisation de cette convolution FFT permet de réduire fortement le temps de calcul, celui-ci passant de plusieurs minutes à quelques secondes.

La méthode *overlap-add* consiste à découper le signal en blocs, à convoluer chaque bloc avec la réponse impulsionnelle du filtre à l'aide de FFT, puis à additionner les parties qui se chevauchent afin de reconstruire le signal filtré complet. Cette approche évite de réaliser individuellement toutes les multiplications entre les échantillons du signal et les coefficients du filtre et devient particulièrement avantageuse lorsque le filtre est de grande longueur.

Le filtre obtenu possède une réponse impulsionnelle symétrique, une phase linéaire. Pour un nombre impair N de coefficients, le retard de groupe est constant. L'application compense ce retard en recadrant le résultat de la convolution afin de conserver un signal filtré aligné temporellement avec le signal d'origine. Cette compensation corrige le décalage temporel introduit par le filtre.

Enfin, le filtrage est réalisé sur l'intégralité du signal ré-échantillonné avant l'extraction de la fenêtre temporelle souhaitée.

4.2 Indicateurs statistiques par fenêtre glissante

Les cinq indicateurs statistiques disponibles dans l'application caractérisent la distribution d'amplitude du signal, calculée sur des fenêtres glissantes successives (durée et recouvrement paramétrables, 5 s et 90 % de recouvrement par défaut) afin d'obtenir une mesure résolue dans le temps plutôt qu'une valeur unique sur l'ensemble de l'enregistrement. Ces indicateurs suivent le formalisme proposé par Antoni et al. [5], qui construit des indicateurs de détection optimaux (au sens de Neyman-Pearson) à partir du rapport de vraisemblance entre un état sain et un état légèrement perturbé, modélisés par une gaussienne généralisée. Chaque indicateur $I(x)$ est normalisé de sorte à valoir 0 pour un signal gaussien classique, correspondant à l'absence d'événement particulier, et à s'écarter de 0 en présence d'un phénomène caractéristique.

4.2.1 Kurtosis

Le kurtosis quantifie la part de la variance du signal portée par des valeurs extrêmes peu fréquentes. Une valeur élevée traduit la présence d'événements impulsifs : chocs, impacts ponctuels et brefs. Il est normalisé par sa variance asymptotique sous hypothèse gaussienne :

$$I_x^{\text{kurtosis}} = \frac{(\text{kurt}(x) - 3)^2}{48} \quad (4.7)$$

où $\text{kurt}(x)$ désigne le kurtosis du signal x sur la fenêtre considérée, et où la valeur 3 correspond au kurtosis théorique d'un signal gaussien : c'est donc l'écart à cette valeur de référence qui est mesuré. Le facteur 48 normalise cet écart par sa variance asymptotique, de sorte que I_x^{kurtosis} reste comparable d'une fenêtre à l'autre. Le kurtosis utilisé dans l'équation (4.7) est défini par le rapport entre le moment d'ordre 4 et le carré du moment d'ordre 2 :

$$s_{4,2}(x) = \frac{\langle |x_n|^4 \rangle}{\langle x_n^2 \rangle^2} \quad (4.8)$$

Cette expression correspond au cas particulier $\beta = 4$, $p = 2$ du formalisme général d'Antoni et al. [5].

4.2.2 Négentropie et synentropie

La négentropie mesure, plus largement que le kurtosis, l'écart de la distribution du signal à une distribution gaussienne, sans se limiter aux seuls événements extrêmes : elle capte aussi des changements plus progressifs de la forme du signal. L'indicateur est défini par :

$$I_x^{\text{négentropie}} = \frac{(\hat{H}_2(x) + \ln 2 + \gamma - 2)^2}{3\pi^2 - 7}, \quad \hat{H}_2(x) = \text{mean} \left(\frac{x^2}{\langle x^2 \rangle} \ln \frac{x^2}{\langle x^2 \rangle} \right) \quad (4.9)$$

où γ est la constante d'Euler, $\langle x^2 \rangle$ la puissance moyenne du signal sur la fenêtre, et $\hat{H}_2(x)$ la négentropie de $|x|^2$. Cette expression correspond au cas $p = 2$ de l'indicateur de négentropie d'ordre p défini par Antoni et al. [5].

Une variante de cet indicateur, la synentropie, est également implémentée dans l'application, disponible comme indicateur optionnel au même titre que le facteur de forme et le facteur de crête (cf. section suivante). Elle repose sur la même idée, mesurer l'écart de $|x|^2$ à son comportement gaussien, mais avec une pondération différente des écarts :

$$I_x^{\text{synentropie}} = \frac{(s - 2)^2}{\pi^2 - 4}, \quad s = \text{mean} \left(\left(\frac{x^2}{\langle x^2 \rangle} - 1 \right) \ln \frac{x^2}{\langle x^2 \rangle} \right) \quad (4.10)$$

La différence entre les deux tient à la pondération appliquée à chaque écart : un facteur $u = x^2/\langle x^2 \rangle$ pour la négentropie (équation (4.9)) contre un facteur $(u - 1)$ pour la synentropie. En pratique, la synentropie reste donc un indicateur proche de la négentropie plutôt qu'une mesure fondamentalement différente.

Kurtosis et négentropie sont complémentaires plutôt que redondants : un événement bref et isolé fait fortement réagir le kurtosis sans nécessairement déplacer la négentropie de façon notable, tandis qu'un changement plus étendu du comportement du signal peut se traduire sur la négentropie sans produire de pic de kurtosis. C'est pour cette raison que ces deux indicateurs sont affichés par défaut dans l'application, et combinés pour l'analyse.

4.2.3 Facteur de forme et facteur de crête

Deux indicateurs supplémentaires sont disponibles dans l'application, masqués par défaut derrière la même case à cocher que la synentropie. Le facteur de forme compare la valeur moyenne absolue du signal à sa valeur efficace (RMS) :

$$I_x^{\text{forme}} = \frac{(\hat{s} - \sqrt{2/\pi})^2}{2(1 - 3/\pi)}, \quad \hat{s} = \frac{\text{mean}(|x|)}{\text{rms}(x)} \quad (4.11)$$

et le facteur de crête compare la valeur efficace au maximum absolu du signal sur la fenêtre :

$$I_x^{\text{crête}} = \frac{45}{8} \left(\sqrt{\frac{\text{rms}(x)}{\max(|x|)}} - \frac{1}{3} \right)^2 \quad (4.12)$$

Le facteur de forme correspond au cas $\beta = 1$, $p = 2$, et le facteur de crête à la limite $p \rightarrow \infty$ avec $\beta = 2$, du même formalisme [5].

Ces deux indicateurs restent basés sur le même type de ratio d'amplitude que le kurtosis, la négentropie et la synentropie : en pratique, ils évoluent de façon proche de ces derniers sans apporter d'information supplémentaire pour le suivi d'une synthèse mécano-chimique. Le facteur de crête est de plus particulièrement sensible à l'échantillon de plus forte amplitude de chaque fenêtre, ce qui le rend peu robuste pour un suivi en continu : un unique échantillon aberrant suffit à faire varier l'indicateur sans que cela traduise un phénomène réel. Ces trois indicateurs optionnels sont conservés dans l'application par souci de complétude et de comparaison, mais restent en retrait de l'analyse par défaut.

4.2.4 Robustesse au changement de fenêtre de calcul

Une attention particulière est portée à l'indépendance des indicateurs vis-à-vis de la durée de la fenêtre d'observation choisie : la normalisation de chaque indicateur (cf. équations (4.7) à (4.12)) est conçue pour rester valide quelle que soit cette durée, ce qui permet à l'utilisateur de comparer des résultats obtenus avec des réglages différents, par exemple des fenêtres de 30 s pour se rapprocher de la résolution temporelle d'une mesure Raman.

Enfin, l'application avertit l'utilisateur lorsque les indicateurs affichés ont été calculés avec un réglage de *crop* différent de celui actuellement actif dans la barre latérale (cf. section 3.2.1), afin d'éviter de présenter des résultats obtenus sur une portion du signal différente de celle actuellement sélectionnée.

4.3 Méthode énergétique

Le spectrogramme représente l'évolution de la distribution fréquentielle du signal en fréquence au cours du temps. Il est calculé par transformée de Fourier à court terme (*Short-Time Fourier Transform*, STFT) : le signal est découpé en blocs de données successifs, éventuellement recouvrants, sur lesquels une transformée de Fourier est appliquée après pondération par une fenêtre. L'application s'appuie sur `scipy.signal.ShortTimeFFT` [6], configurée pour renvoyer directement une estimation de densité spectrale de puissance (PSD) :

$$\text{Spgx}(f, t) = |\text{STFT}\{x\}(f, t)|^2 \quad (4.13)$$

Plusieurs paramètres de calcul sont réglables depuis l'interface : le type de fenêtre de pondération (Hanning, Hamming, rectangulaire, Blackman), la taille de la fenêtre en échantillons (qui détermine le compromis entre résolution temporelle et résolution fréquentielle), le taux de recouvrement entre fenêtres consécutives, et un facteur de zéro-padding qui augmente artificiellement la résolution fréquentielle affichée sans ajouter d'information réelle au signal.

À partir du spectrogramme, l'application permet de suivre l'évolution temporelle de l'énergie acoustique contenue dans une bande de fréquences donnée. Pour une bande $[f_{\min}, f_{\max}]$, l'énergie est obtenue en sommant la densité spectrale de puissance sur les indices de fréquence compris dans la bande, à chaque instant du spectrogramme :

$$E(t) = \sum_{f_{\min} \leq f \leq f_{\max}} \text{Spgx}(f, t) \quad (4.14)$$

Cette énergie est ensuite normalisée entre 0 et 1 sur l'ensemble de l'enregistrement, pour faciliter la comparaison entre bandes de niveaux d'énergie différents :

$$\tilde{E}(t) = \frac{E(t) - \min_t E(t)}{\max_t E(t) - \min_t E(t)} \quad (4.15)$$

Contrairement aux indicateurs statistiques, l'analyse par bandes d'énergie exploite un paramètre connu du dispositif expérimental : la fréquence de rotation du broyeur, fixée par l'utilisateur au moment de programmer la synthèse. Cette rotation se traduit dans le signal acoustique par une composante répétée à la fréquence f_0 et se retrouve, avec une énergie différente, à ses harmoniques $2f_0$, $3f_0$, etc. Suivre l'énergie sur f_0 et ses premières harmoniques permet ainsi de cibler spécifiquement la signature mécanique du broyage plutôt que d'intégrer indistinctement sur l'ensemble du spectre.

Deux modes de définition des bandes sont proposés dans l'application. L'utilisateur renseigne une fréquence fondamentale f_0 (par exemple liée à une vitesse de rotation du dispositif de broyage) ainsi qu'une demi-largeur de bande δf . L'application calcule alors automatiquement les bandes correspondant aux harmoniques successives de f_0 : $H_0 = f_0$, $H_1 = 2f_0$, $H_2 = 3f_0$, etc., chacune définie comme $[n \cdot f_0 - \delta f, n \cdot f_0 + \delta f]$, et ne conserve que les harmoniques dont la fréquence centrale reste inférieure à la fréquence de Nyquist du signal ré-échantillonné. L'utilisateur sélectionne ensuite librement les harmoniques à afficher parmi celles-ci.

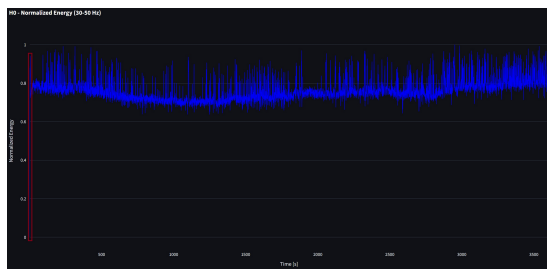
En complément, l'utilisateur peut définir un nombre arbitraire de bandes de fréquences manuelles, sans lien avec une fondamentale, utile pour explorer une zone du spectre sans hypothèse préalable sur sa structure harmonique.

Dans les deux cas, chaque bande affichée est mise en cache dans `session_state` sous une clé dérivée de ses bornes de fréquence, de l'orientation choisie pour les axes (oui, aussi étrange que cela puisse paraître les chimistes s'emploient à positionner le temps sur l'axe Y...) et du facteur de décimation d'affichage (cf. section 3.2.1) : rouvrir une page déjà visitée n'entraîne donc pas de recalcul si les réglages n'ont pas changé, et les bandes qui ne sont plus sélectionnées sont automatiquement retirées du cache pour ne pas accumuler de figures obsolètes.

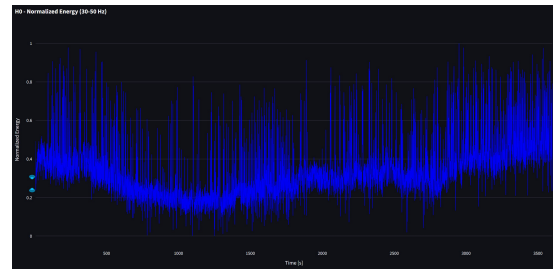
La fonction `ShortTimeFFT` ajoute automatiquement des zéros aux deux extrémités du signal, indépendamment du facteur de zéro-padding choisi par l'utilisateur, afin de pouvoir calculer une valeur de spectrogramme même pour les toutes premières et dernières fenêtres. Cet ajout artificiel produit des colonnes de spectrogramme partiellement calculées sur du silence, qui se traduisent par des valeurs d'énergie anormalement faibles aux extrémités temporelles. Ce comportement est identifié en cours de stage sous la forme de « barres latérales » parasites sur les graphes d'énergie normalisée (cf. section 4.3) :

la normalisation min/max de l'énergie amplifie ce défaut de bord en le ramenant artificiellement à 0. La correction retenue consiste à retirer explicitement les colonnes du spectrogramme correspondant à ces zones polluées par les zéros ajoutés, en ne conservant que les fenêtres entièrement comprises dans le signal d'origine : cette correction est appliquée automatiquement par l'application, sans action requise de l'utilisateur.

Des barres similaires peuvent néanmoins subsister pour une raison différente, cette fois liée au contenu réel du signal et non à un artefact de calcul : en pratique, le microphone est le plus souvent lancé manuellement quelques instants avant l'allumage effectif du dispositif de broyage, et parfois arrêté quelques instants après ; les premières et dernières secondes du fichier `.wav` correspondent alors à du silence ou à un bruit ambiant réel, sans lien avec le broyage. Ce silence produit une énergie normalisée effectivement basse à ces instants, que la correction précédente ne peut pas retirer puisqu'il s'agit cette fois d'un contenu légitime du signal, et non d'un artefact introduit par le calcul. Un léger *crop* du signal (cf. figure 4.5), pour exclure ces marges d'enregistrement non pertinentes, est alors nécessaire pour retrouver une énergie normalisée cohérente sur l'ensemble de la plage temporelle affichée.



(a) Sans *crop* : artefacts relevés en rouge.



(b) Après *crop* : artefacts supprimés.

FIGURE 4.5 – Effet du *crop* sur les artefacts de bord de l'énergie normalisée par bande de fréquence, dans le cas où le signal contient encore quelques secondes enregistrées avant l'allumage du dispositif (ou après son arrêt). Sans *crop* (a), ce silence réel en début et fin d'enregistrement se traduit par une énergie anormalement faible à ces instants. Un léger *crop* du signal (b), pour exclure ces marges non pertinentes, suffit à retrouver une énergie normalisée cohérente sur l'ensemble de la plage temporelle.

Un spectrogramme calculé sur un enregistrement long, avec une taille de fenêtre fine et un facteur de zéro-padding élevé, peut produire une matrice de plusieurs millions de points : son affichage brut saturerait le rendu graphique. L'application applique donc une décimation d'affichage adaptative, distincte du paramètre de décimation partagé décrit en section 3.2.1 : le nombre de points affichés en temps et en fréquence est automatiquement borné, indépendamment des plages sélectionnées par l'utilisateur via les curseurs de zoom. Le calcul sous-jacent, lui, reste inchangé : seul l'affichage est simplifié, et redevient pleinement détaillé dès que l'utilisateur resserre la plage de temps ou de fréquence observée.

4.4 Données complémentaires de température

En complément du signal acoustique, l'application permet de charger un fichier de température au format `.dat`, produit par la caméra thermique du dispositif expérimental (cf. section 2.2). Le format de ces fichiers n'étant pas strictement normalisé, le séparateur décimal, le délimiteur de colonnes et l'encodage du fichier sont paramétrables depuis l'interface, avec des valeurs par défaut correspondant à la convention actuellement en usage à l'ICGM. Le fichier est analysé pour en extraire un en-tête de métadonnées (date, heure, unité, résolution) ainsi qu'un ensemble de colonnes de température, chacune correspondant à une zone du réacteur suivie par la caméra thermique ; les lignes ou valeurs qui ne peuvent pas être interprétées sont comptabilisées et signalées à l'utilisateur, plutôt que de produire une erreur silencieuse.

Le temps déclaré par le fichier de température est directement en secondes écoulées depuis le début de l'enregistrement, c'est-à-dire dans le même référentiel absolu que le signal acoustique. La même fenêtre de *crop* que celle appliquée au signal acoustique (cf. section 3.2.1) est donc directement transposable à la température, sans nécessiter de recalage manuel : si la fenêtre de *crop* active ne recouvre pas la période couverte par l'enregistrement de température, l'application le signale et retombe sur l'affichage de la température complète plutôt que d'afficher un graphe vide.

L'intérêt principal de cette page est de permettre l'affichage simultané de la température et de tout indicateur déjà calculé dans l'application (indicateurs statistiques, énergie par bande), sous forme de sous-graphes partageant le même axe temporel, afin de faciliter la recherche visuelle de corrélations entre événements thermiques et événements acoustiques. Une zone temporelle particulière peut également être mise en évidence simultanément sur l'ensemble des sous-graphes affichés, pour annoter un événement d'intérêt repéré par l'utilisateur.

4.5 Export et traçabilité

La dernière étape de la pipeline permet d'exporter les figures et données calculées au cours des étapes précédentes, dans un unique fichier `.zip` contenant les formats sélectionnés (PNG, JPG, HTML interactif, CSV) ainsi qu'un rapport de traitement.

Plutôt que de coder individuellement, sur la page d'export, la logique propre à chaque type de figure (signal, indicateur, spectrogramme, bande d'énergie, température...), l'application construit dynamiquement la liste des éléments exportables à partir d'un registre centralisé (cf. section 3.2) : chaque élément disponible déclare simplement sa catégorie, la fonction permettant de reconstruire sa figure, et celle permettant de reconstruire son

DataFrame pour l'export CSV. La page d'export se contente de parcourir ce registre pour afficher les cases à cocher correspondantes ; ajouter un nouveau type d'élément exportable ne nécessite donc pas de modifier la page elle-même.

Pour les exports CSV, un facteur de décimation indépendant de celui utilisé pour l'affichage à l'écran (cf. section 3.2.1) peut être appliqué : il ne réduit que la taille du fichier exporté, sans jamais affecter les valeurs elles-mêmes, qui restent calculées à pleine résolution. Les cinq derniers points de chaque série sont systématiquement conservés lors de la décimation, pour ne pas perdre la borne de fin d'un tracé.

Un fichier `processing_report.txt` est systématiquement inclus dans l'export, quels que soient les formats sélectionnés. Il consigne l'ensemble des paramètres utilisés à chaque étape de la pipeline : fichier source, réglage de *crop*, facteur de sous-échantillonnage, paramètres de filtrage (type, méthode de conception directe ou IFIR, fréquences de coupure), paramètres du spectrogramme, bandes d'énergie actives, et fenêtre de calcul utilisée pour chaque indicateur statistique. Ce rapport répond directement à l'absence de vérité terrain identifiée en section 2 : à défaut de pouvoir comparer un résultat à une référence absolue, il garantit au moins que chaque figure exportée reste traçable jusqu'aux paramètres exacts qui l'ont produite, condition nécessaire à la reproductibilité d'une analyse.

5. Résultats et validation sur cas réel

5.1 La pipeline de A à Z

De manière à mettre en exergue le produit du stage, il est possible de détailler la pipeline de traitement de l'application de surveillance acoustique sur un signal réel proposé par les mécanochimistes de l'ICGM. En effet, le développement d'une application nécessitant des phases de test régulières, plusieurs signaux sont mis à disposition par l'équipe de Montpellier.

L'exemple porte sur une synthèse **TA-DABCO** du *29/06/2026*, après correction du problème de saturation du microphone.

5.1.1 Chargement et visualisation du signal brut

La première étape consiste à charger le fichier `.wav` depuis la page `Upload`. Conformément au fonctionnement décrit en section 4.1.2, seule la portion du signal effectivement affichée à l'écran est convertie en tableau exploitable à ce stade, ce qui permet une visualisation immédiate du signal brut malgré sa taille, avant toute opération de sous-échantillonnage.

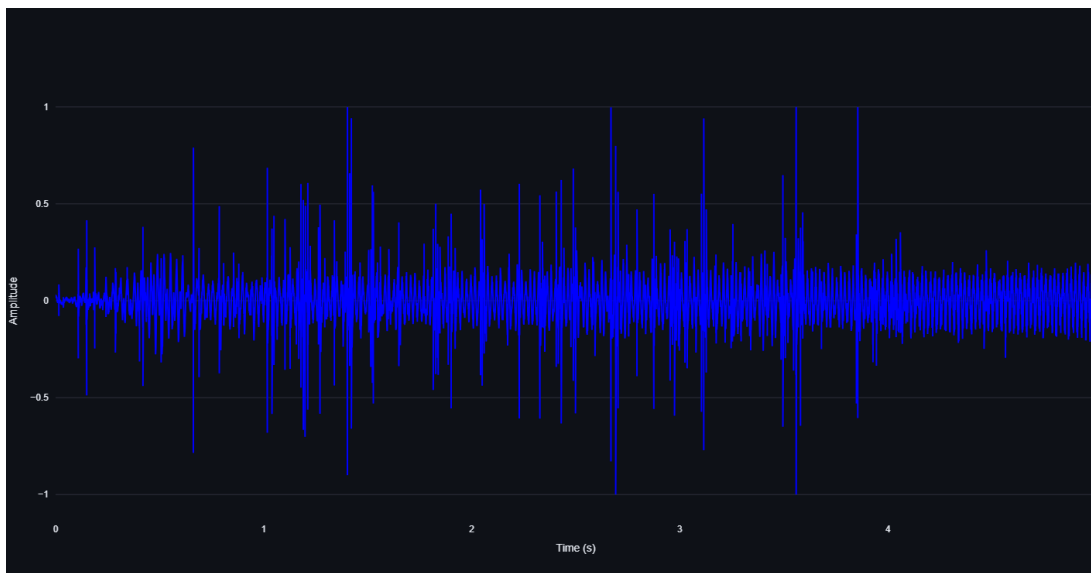


FIGURE 5.1 – Page `Upload` après chargement du signal brut de la synthèse TA-DABCO du *29/06/2026*.

5.1.2 Sous-échantillonnage et filtrage passe-haut

Le signal est ensuite sous-échantillonné d'un facteur 44, conformément au réglage par défaut de l'application (cf. section 4.1.2), ramenant la fréquence d'échantillonnage aux alentours de 1 kHz. Un filtrage passe-haut est appliqué sur le signal sous-échantillonné, avec les paramètres suivants : gain unitaire, fréquence de bande passante $f_{\text{pass}} = 40$ Hz et fréquence de bande atténuée $f_{\text{stop}} = 10$ Hz. Ce filtrage vise à supprimer le bruit environnemental basse fréquence, comme illustré en section 4.1.2 sur un autre signal.

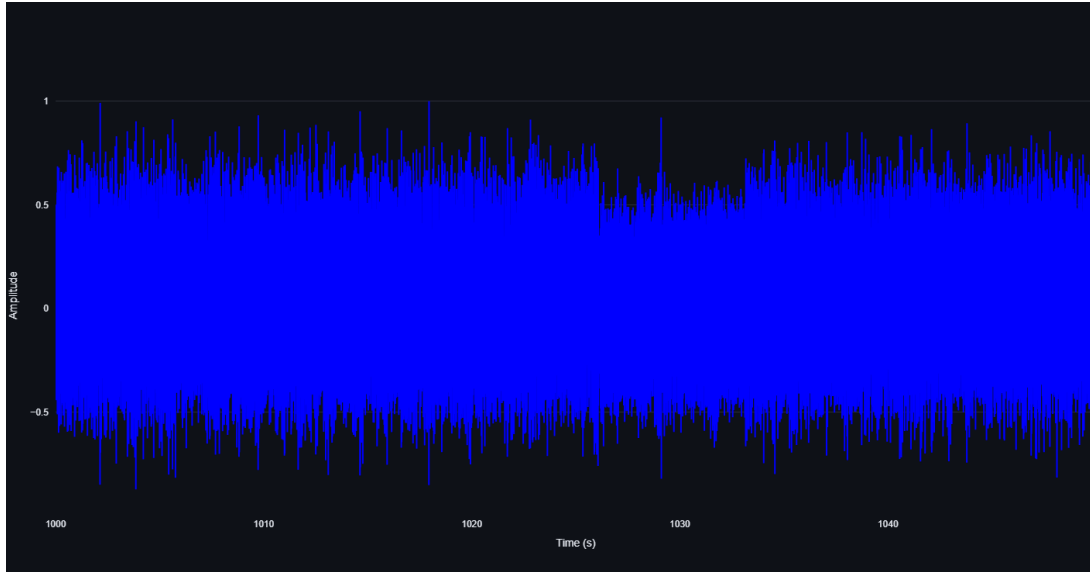


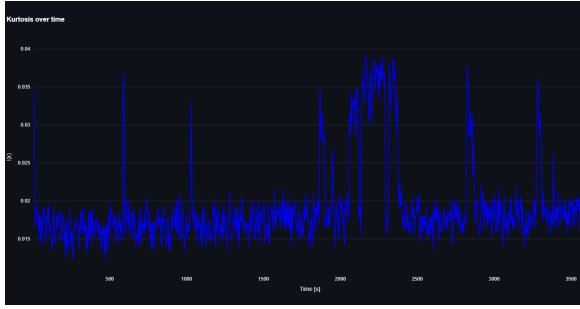
FIGURE 5.2 – Signal sous-échantillonné (facteur 44) puis filtré (passe-haut, $f_{\text{stop}} = 10$ Hz, $f_{\text{pass}} = 40$ Hz, gain = 1) pour la synthèse TA-DABCO.

5.1.3 Recadrage temporel du signal

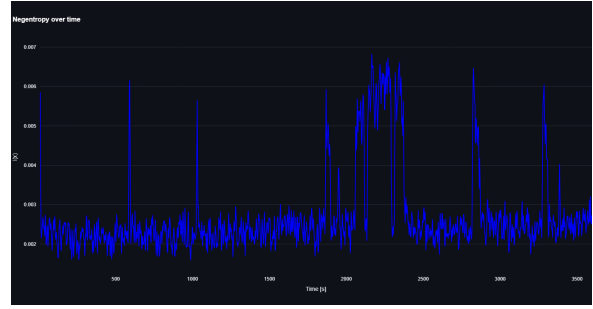
Un *crop* est ensuite appliqué sur la plage [5 s - 3600 s], via la barre latérale partagée décrite en section 3.2.1. Ce recadrage permet d'exclure les premières secondes de l'enregistrement, généralement antérieures à l'allumage effectif du dispositif de broyage, et d'écarter la fin de la piste au-delà de la durée réelle de synthèse. Ce réglage est ensuite pris en compte automatiquement par l'ensemble des pages en aval de la pipeline.

5.1.4 Indicateurs statistiques : kurtosis et négentropie

Le kurtosis et la négentropie sont calculés par fenêtre glissante de 5 secondes sur le signal préprocessé (sous-échantillonné, filtré et recadré), selon le formalisme détaillé en section 4 (équations (4.7) et (4.9)). Ces deux indicateurs, affichés par défaut dans l'application, permettent de repérer respectivement les événements impulsifs ponctuels et les changements plus progressifs de la distribution du signal.



(a) Kurtosis.



(b) Négentropie.

FIGURE 5.3 – Kurtosis et négentropie calculés sur le signal préprocessé de la synthèse TA-DABCO.

5.1.5 Spectrogramme et bandes d'énergie harmoniques

Le spectrogramme du signal préprocessé est calculé en utilisant une fenêtre de Hanning de 5 secondes avec un facteur de *zero-padding* de 4 et un paramètre d'*overlap* de 50 % selon la méthode décrite en section 4.3 (équation (4.13)). À partir de ce spectrogramme, quatre bandes d'énergie sont définies autour de la fréquence fondamentale $f_0 = 50$ Hz et de ses trois premières harmoniques (H_0 à H_3 : 50, 100, 150 et 200 Hz), chacune avec une demi-largeur de bande $\Delta f = 10$ Hz, conformément au mécanisme justifié en section 4.3.

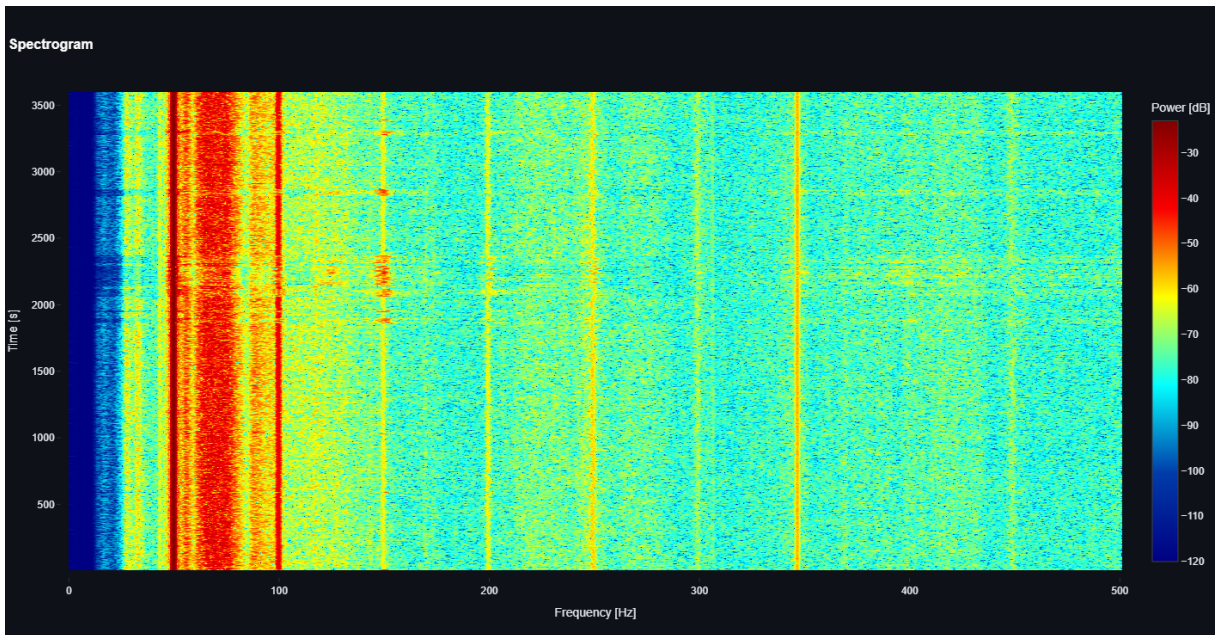


FIGURE 5.4 – Spectrogramme du signal préprocessé, synthèse TA-DABCO. Les harmoniques du signal apparaissent clairement sous forme de barres verticales aux multiples de la fréquence de broyage de 50 Hz

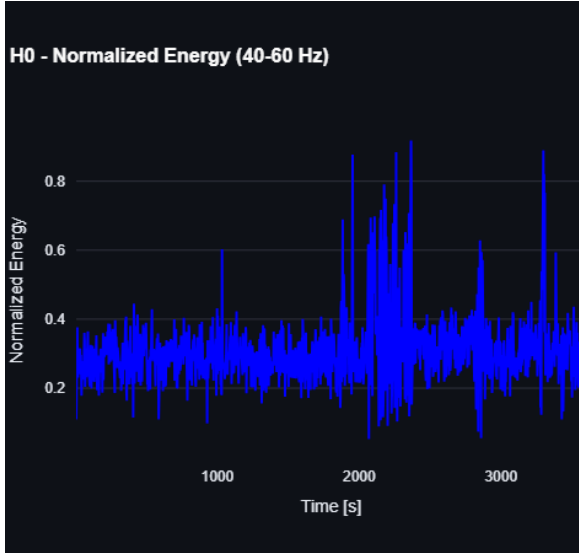
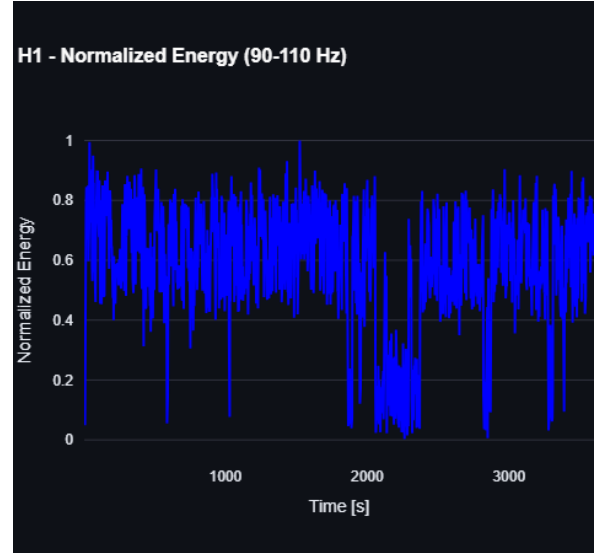
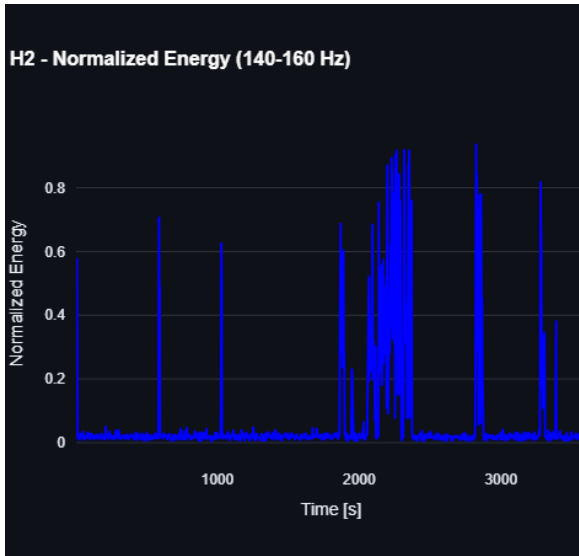
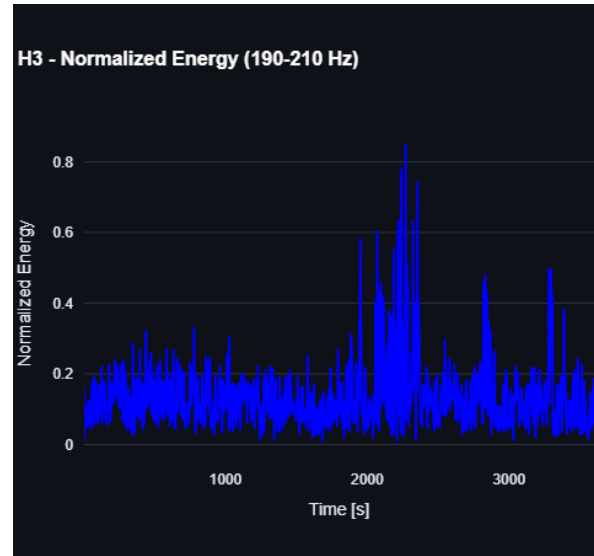
(a) H_0 (50 Hz ± 10 Hz).(b) H_1 (100 Hz ± 10 Hz).(c) H_2 (150 Hz ± 10 Hz).(d) H_3 (200 Hz ± 10 Hz).

FIGURE 5.5 – Énergie normalisée sur les bandes H_0 à H_3 (50, 100, 150 et 200 Hz, ± 10 Hz), synthèse TA-DABCO.

5.1.6 Ajout de la température et mise en parallèle des indicateurs

Le fichier de température associé à la synthèse TA-DABCO est ensuite chargé depuis la page `AdditionalData`, selon la procédure décrite au chapitre 4 (données complémentaires de température). Le temps déclaré par ce fichier étant exprimé dans le même référentiel que le signal acoustique, la fenêtre de *crop* appliquée précédemment (5 s – 3600 s) s'applique directement à l'affichage de la température.

La dernière étape consiste à afficher simultanément, sous forme de sous-graphes partageant le même axe temporel, la température, l'énergie normalisée des bandes H_0 et H_1 , ainsi que la négentropie calculée précédemment. Une zone temporelle d'intérêt est mise en évidence (*highlight*) simultanément sur l'ensemble de ces sous-graphes, afin de faciliter la recherche visuelle d'une corrélation entre l'événement thermique observé et l'évolution des indicateurs acoustiques sur cette même période.

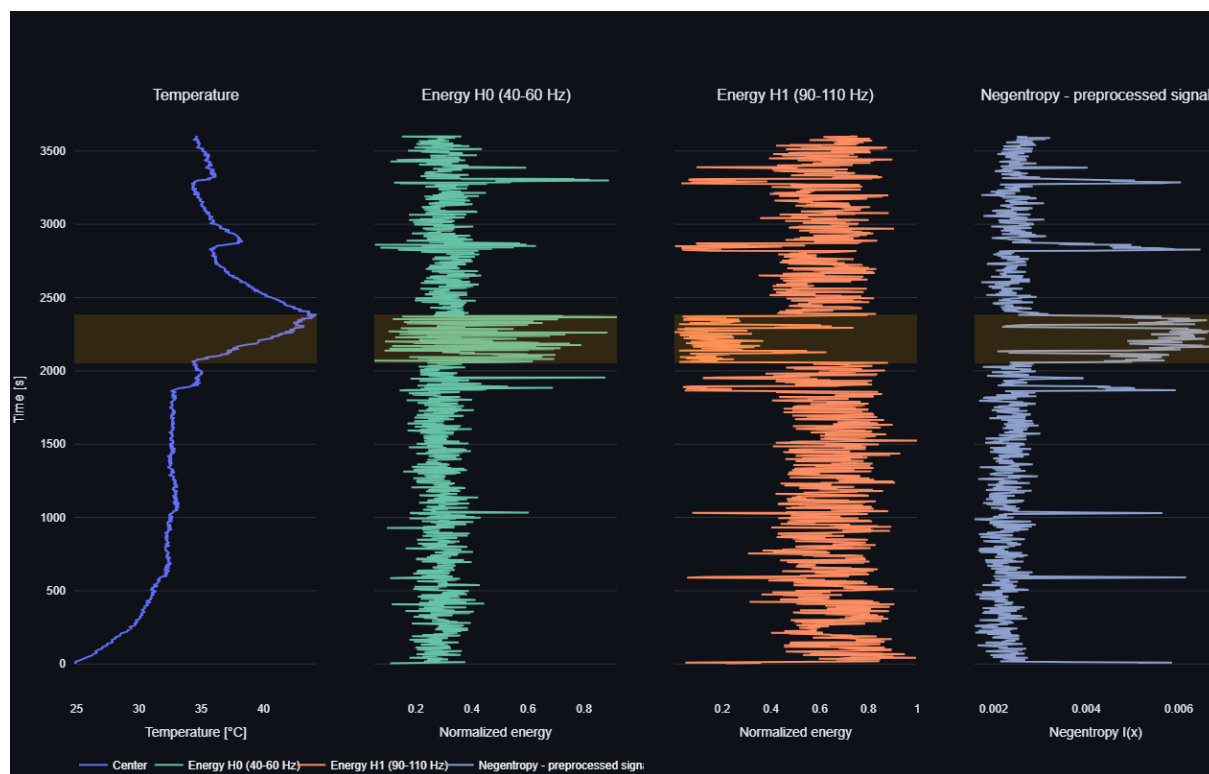


FIGURE 5.6 – Mise en parallèle de la température, de l'énergie normalisée des bandes H_0 et H_1 , et de la négentropie, avec mise en évidence d'une zone temporelle d'intérêt, synthèse TA-DABCO.

Il revient alors au mécanochimiste de confronter la zone temporelle identifiée grâce à l'application à une spectroscopie Raman pour tirer des conclusions adéquates au sujet de la réaction. L'exemple proposé ici permet de mettre en évidence le fonctionnement de l'application, la pertinence des indicateurs et figures extraites et l'ergonomie générale du produit.

5.2 Retours d'usage

L'exécution d'une pipeline complète concerne directement le développeur, qui doit faire des choix d'ergonomie et d'implémentation, mais le produit final est destiné aux mécanochimistes : ce sont eux qui savent quelles informations l'application doit leur permettre d'extraire, et de quelle manière. Dans cette logique, discuter de l'usage qu'en font les

principaux concernés se révèle tout aussi pertinent que de discuter du code produit et de son exécution.

Pour cela, on se réfère directement aux retours d’usage formulés par l’équipe de l’ICGM en cours de développement de l’application, mais surtout en fin de stage, autour de la V7 faisant office de produit fini (cf. tableau 3.1).

De manière générale, aucune fonctionnalité n’est retirée de l’application au fil du développement : chaque outil implémenté l’est sur la base d’une pertinence déjà validée en amont (à l’exception des indicateurs statistiques autres que le kurtosis), ce qui explique la stabilité globale du périmètre fonctionnel d’une version à l’autre. Les retours des utilisateurs portent alors moins sur le *quoi* que sur le *comment* : comment rendre une fonctionnalité déjà présente plus lisible, plus rapide à prendre en main, ou plus adaptée à l’usage qu’ils en ont.

Une partie de ces retours concerne des anomalies de fonctionnement, identifiées et corrigées au fil des versions. C’est par exemple le cas d’un défaut lié au recadrage temporel du signal, remonté par l’équipe après plusieurs semaines d’utilisation : dans une configuration particulière, le signal filtré n’est pas mis à jour après modification du *crop*. Le correctif associé est livré en quelques jours sous la forme d’une version intermédiaire (V6.1), avant confirmation par l’équipe que le problème est résolu.

D’autres retours relèvent davantage de suggestions ergonomiques, formulées après une prise en main prolongée de l’outil. Par exemple, inverser l’ordre des pages **StatisticalIndicators** et **EnergyMethod**, l’usage courant consistant à consulter en premier les bandes d’énergie harmoniques, au risque de laisser de côté une information pourtant disponible dans les indicateurs statistiques ; cette réorganisation est intégrée dans la V7. De la même manière, il est suggéré de remplacer le curseur de facteur de sous-échantillonnage par un menu déroulant affichant directement la fréquence d’échantillonnage résultante, jugé plus explicite pour un public non familier du traitement du signal ; cette modification n’est en revanche pas retenue, le gain en clarté ne compensant pas le coût en calculs et en encombrement de l’interface qu’elle implique.

Les modifications effectivement intégrées concernent ainsi principalement l’ajout d’info-bulles sur les algorithmes de traitement du signal exploités, pour permettre une utilisation fluide de l’application sans en maîtriser les fondements théoriques, ainsi que des ajouts de figures et d’outils non techniques mais ergonomiques (superposition de graphes, mise en évidence d’une zone temporelle, export au format JPG). Le but est de permettre à un utilisateur à la fois non formé à Python et non formé aux algorithmes de traitement du signal employés d’exploiter l’application à son plein potentiel.

La réactivité apportée à ces retours, à travers le rythme hebdomadaire de livraison (cf. section 3.1), permet de limiter le nombre de défauts non résolus d’une version à l’autre, la

V7 fait office de livrable stable (avec probablement des défauts à corriger ultérieurement) sur le dépôt. Ce constat, associé aux retours finaux de l'équipe soulignant la qualité du travail produit, confirme que l'application répond dans l'ensemble aux besoins exprimés tout au long du stage.

5.3 Cas d'usage : mise en évidence d'un intermédiaire réactionnel

Au-delà des retours d'usage généraux discutés précédemment, il est intéressant d'illustrer l'application à travers un exemple d'exploitation concret, mené en autonomie par un utilisateur final. On prend ici pour exemple le travail d'Assane Goumble, stagiaire de M2 à l'ICGM, dont le sujet de recherche porte sur l'identification d'une réaction de mécanochimie présentant un intermédiaire réactionnel. Il exploite l'application dans son usage quotidien pour extraire et traiter rapidement les données acoustiques issues de ses synthèses.

L'exemple présenté ici porte sur une réaction entre la phénazine (Phen) et le 1,4-diiodotétrafluorobenzène (TFIB), pour laquelle un suivi acoustique d'une heure de broyage horizontal à 30 Hz est réalisé, selon le dispositif expérimental décrit en section 2.2. Les fichiers CSV des indicateurs énergétiques et statistiques sont extraits depuis la page **Export** de l'application (cf. section 4, export et traçabilité), puis mis en parallèle avec les résultats de spectroscopie Raman obtenus indépendamment sur le même essai, afin de mener le travail d'analyse et de présentation aux pairs. Cette mise en parallèle, actuellement réalisée hors de l'application faute d'intégration du signal Raman à la pipeline (cf. section 2.2), constitue une piste d'évolution possible évoquée en perspective de ce rapport.

La figure 5.7 illustre un essai réalisé dans ces conditions, constituant une preuve de concept quant à la possibilité de détecter par suivi acoustique la présence d'un intermédiaire réactionnel. Elle met en parallèle, de gauche à droite : la température, le signal Raman, l'énergie normalisée de la bande de fréquence 140–160 Hz et le kurtosis calculé sur le signal préprocessé. Trois zones temporelles sont mises en évidence sur l'ensemble de ces sous-graphes, correspondant à trois régimes successifs de la réaction : une phase initiale (A+B), une phase intermédiaire (C) associée à la formation de l'intermédiaire réactionnel, et une phase finale (D). Les transitions entre ces trois zones, où l'évolution conjointe des indicateurs acoustiques et du signal Raman est la plus marquée, constituent un objet d'étude à part entière, que la fonctionnalité de *highlight* synchronisé décrite en section 4 permet d'isoler directement dans l'application.

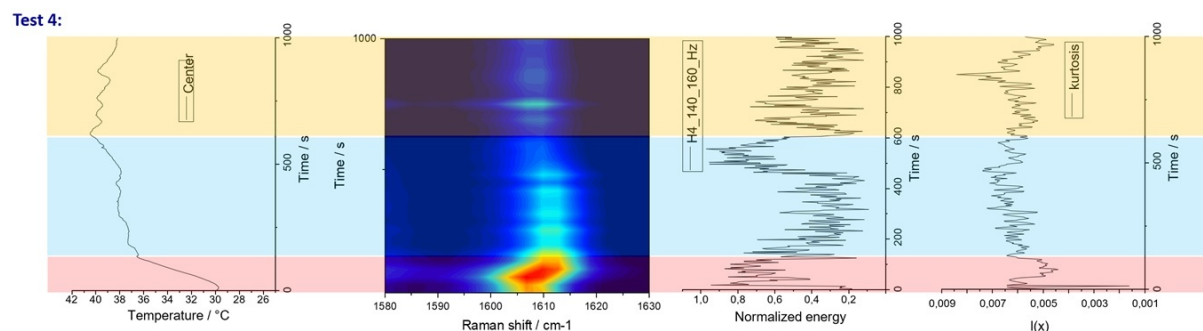


FIGURE 5.7 – Suivi d’une synthèse Ph-TFIB par broyage horizontal à 30 Hz (1 heure). De gauche à droite : température, signal Raman, énergie normalisée de la bande 140–160 Hz, kurtosis. Les zones A+B, C et D correspondent à trois régimes successifs identifiés de la réaction, la zone C étant associée à la présence d’un intermédiaire réactionnel.

Cet exemple illustre la manière dont l’application permet à un chimiste de suivre le déroulement d’une synthèse à partir d’un signal sonore dûment exploité. Le contenu acoustique des deux graphes de droite est en adéquation avec les observations que le mécanochimiste peut effectuer grâce à la spectroscopie Raman et à la courbe de température.

Pour le kurtosis, il reste faible dans les premières minutes de l’enregistrement, ce qui traduit un broyage homogène sans événement remarquable. Une augmentation nette et soudaine indique un changement de régime, probablement lié à la formation de l’intermédiaire réactionnel. Le régime évolue une dernière fois lors de la phase finale, où le kurtosis diminue à nouveau.

Il en va de même pour l’énergie normalisée, dont les phases de croissance et de décroissance traduisent des modifications du régime de contact entre les billes, les réactifs et le réacteur. En fonction de la texture du mélange, ces variations permettent de distinguer clairement les différentes phases d’intérêt de la réaction.

6. Enseignements et développement de compétences

6.1 Aspects techniques

Une application web en Python

L'essentiel des enseignements tirés de ce stage de 11 semaines au sein du GIPSA-lab concerne la montée en autonomie en développement Python. Le travail livré représente plus de 4000 lignes de code réparties dans une vingtaine de fichiers interdépendants, qui constituent ensemble une application de surveillance acoustique exploitable par l'équipe de chimistes de l'ICGM.

Streamlit épargne une partie du travail habituellement nécessaire pour séparer front-end et back-end (pas de HTML, CSS ou JavaScript à écrire), mais ne dispense pas d'un travail rigoureux de développeur, à plusieurs niveaux : architecture du code en amont (organisation des modules, dépendances entre fonctions, choix de ce qui doit vivre dans `utils/` plutôt que dans les pages, cf. section 3.2), développement à proprement parler, puis vérification de la conformité du code produit aux attentes, avant d'ajouter une couche propre à ce contexte : la mise en forme de l'interface utilisateur, l'intégration des scripts de calcul aux widgets, et la prise en compte des retours d'usage et la correction des bugs remontés par l'équipe de l'ICGM.

Cette expérience constitue une première approche concrète du développement web et positionne Streamlit comme un outil intéressant pour le quotidien d'un ingénieur en traitement du signal, pour transformer rapidement un script en outil partageable ou en démonstration fluide. Ce stage permet de passer d'un niveau débutant à un niveau intermédiaire sur cet outil.

Indicateurs statistiques et méthodes de traitement du signal

Au-delà du développement applicatif à proprement parler, une bonne partie du stage a porté sur la compréhension et la vérification d'algorithmes de traitement du signal qui ne sont pas toujours triviaux à implémenter. C'est le cas en particulier des indicateurs statistiques (kurtosis, négentropie, synentropie, facteurs de forme et de crête, cf. section 4),

dont la formulation par fenêtre glissante et la normalisation associée demandent une lecture attentive de la littérature pour être reproduites correctement, ainsi que des méthodes liées au filtrage (Remez, IFIR) et à l'analyse temps-fréquence (STFT, bandes d'énergie), où une erreur d'implémentation peut rester invisible tant qu'elle n'est pas confrontée à un cas de validation spécifique.

Ce stage propose un double travail : développer et faire vivre une application, d'une part ; développer, comprendre et vérifier des algorithmes de calcul pas toujours évidents, d'autre part. Les deux exercices sont complémentaires mais mobilisent des réflexes différents : le premier demande de penser en termes de structure et de maintenir, le second en termes d'exactitude de calculs et de cas limites.

Recherche bibliographique et portage de méthodes

Certaines méthodes utilisées dans l'application n'existent pas nativement en Python. C'est notamment le cas de la conception de filtres IFIR (section 4.1.2) : aucune bibliothèque Python de référence ne propose cette fonctionnalité, ce qui nécessite de reconstruire la méthode à partir de la documentation Matlab existante et des articles cités en référence, puis de la traduire et de l'adapter au contexte de l'application (choix automatique du facteur d'étirement, seuil de bascule entre conception directe et IFIR). De la même manière, l'implémentation des indicateurs statistiques demande une lecture bibliographique large, pour comprendre le formalisme général dont ils sont issus avant de pouvoir en vérifier chaque cas particulier.

Ce travail de recherche, en amont du développement, est une part du stage des plus formatrices : il ne s'agit pas seulement d'utiliser une méthode déjà encapsulée dans une bibliothèque, mais de comprendre suffisamment finement son principe pour pouvoir la reconstruire et en garantir la validité.

6.2 Autonomie et conduite de projet

Au-delà des aspects techniques, ce stage contribue au développement d'une forte autonomie de travail. Basé au GIPSA-lab à Grenoble, il n'y a pas d'occasions d'échanger en présentiel avec l'équipe de l'ICGM, à Montpellier, en dehors du déplacement initial (cf. section 2.2). Seul le tuteur de stage est disponible sur place au quotidien : les échanges, généralement organisés une fois par semaine (le mercredi ou le jeudi selon les disponibilités), servent avant tout de point d'ancrage pour orienter les pistes à explorer et de recours en cas de blocage. Le reste du temps, la conduite du projet (choix des priorités, découpage du travail, résolution des problèmes rencontrés) se fait en autonomie.

Cette organisation demande de structurer un planning de travail, en tenant deux objectifs de front chaque semaine : faire progresser l'application avec de nouvelles fonctionnalités, tout en corrigeant les bugs et les points d'ergonomie remontés par les utilisateurs sur la version précédente (cf. cycle de retours hebdomadaire, section 3.1). Ces deux priorités sont parfois en tension : le temps passé à corriger un défaut d'ergonomie signalé est autant de temps non consacré à une nouvelle méthode de traitement, et inversement. Trouver le bon équilibre d'une semaine à l'autre constitue un exercice de gestion de projet à part entière.

Un aspect supplémentaire de cette autonomie tient à la communication avec les utilisateurs finaux eux-mêmes. L'équipe de mécanochimistes de l'ICGM n'a ni le temps ni nécessairement la formation pour entrer dans le détail du code Python ou des algorithmes de traitement du signal sous-jacents. Il est nécessaire d'être en mesure d'expliquer les choix faits dans l'application : ce qu'un réglage modifie concrètement, pourquoi tel indicateur réagit ainsi, ce qui a changé d'une version à l'autre... dans des termes compréhensibles, tout en conservant les aspects techniques pour un futur développeur (cf. section 3.2, documentation). Cette double communication, technique et non technique, constitue un apprentissage à part entière, distinct des compétences purement algorithmiques ou logicielles développées par ailleurs.

6.2.1 Usage de l'IA générative

Le développement de l'application s'appuie ponctuellement sur un outil d'IA générative (Claude Sonnet 5), dont l'usage est restreint à des cas précis et discuté ouvertement avec l'équipe. Le principe directeur est de conserver la maîtrise et la compréhension du code produit : l'IA n'est pas utilisée pour écrire une fonctionnalité de bout en bout, mais pour deux types de tâches identifiées.

Le premier cas d'usage concerne la correction de bugs sur lesquels un temps disproportionné est passé sans parvenir à identifier la cause exacte. Dans ce cas, l'outil sert de second regard sur un problème déjà caractérisé, plutôt que de solution clé en main : le correctif proposé est relu, compris et, si besoin, réécrit pour rester cohérent avec le reste du code, de sorte que sa maîtrise reste acquise avant son intégration.

Le second cas d'usage concerne la factorisation du code une fois une fonctionnalité déjà développée et validée. C'est notamment le travail mené lors des versions V5 et V6 (cf. tableau 3.1), consacrées à un nettoyage général du code : automatisation des éléments redondants entre les six pages de l'application, regroupement de logiques dupliquées dans des fonctions communes (par exemple les fonctions `fct_slider`, `fct_checkbox` et consorts du module `widgets.py`, cf. section 3.2), ou simplification de blocs de code répétitifs à l'aide de bibliothèques mieux adaptées. La logique de traitement reste définie en amont :

l'IA intervient pour réduire un code déjà fonctionnel, sans en changer le comportement. Ce travail de factorisation a permis un gain de l'ordre de plusieurs centaines de lignes de code sur les versions concernées, et une lisibilité accrue de l'application pour sa prise en main ultérieure par l'équipe de l'ICGM.

Le seul point du code sur lequel la maîtrise reste partielle concerne la structure `ExportItem`, brique du registre `export_registry` (cf. section 4, export et traçabilité) qui encapsule, pour chaque élément exportable, sa catégorie, la fonction permettant de reconstruire sa figure et celle permettant de reconstruire son `DataFrame` associé. Cet outil fourre-tout, pensé pour rester générique afin d'accueillir tout nouveau type d'élément exportable sans modifier la page `Export` elle-même, a été construit avec l'appui de l'IA pour sa partie la plus générique. Sa complexité reste limitée : son principe de fonctionnement est compris et documenté (cf. `INFOS.md`, section 3.2), seule une partie de son écriture initiale a bénéficié de cet appui.

Plus largement, un ingénieur en traitement du signal n'a pas nécessairement connaissance de l'ensemble des pratiques de développement logiciel. Dans ce contexte, l'IA générative constitue également un outil de formation continue, permettant de découvrir de nouvelles façons de structurer un code au fil du développement de l'application, plutôt qu'un raccourci pour éviter d'en écrire. Le même outil est par ailleurs mobilisé ponctuellement dans la rédaction de ce rapport, pour simplifier quelques passages (mise en forme de tableaux ou de figures `LATEX`, reformulation de phrases trop denses), sans intervenir sur le contenu technique ni les résultats présentés.

Conclusion et perspectives

Ce stage est motivé par un objectif de consolider des méthodes existantes de suivi acoustique de synthèses mécano-chimiques et de les intégrer au sein d’une application unique, directement utilisable par l’équipe de mécano-chimistes de l’ICGM sans connaissance préalable en programmation.

Le chapitre 2 pose le cadre scientifique et technique du projet : l’intérêt du suivi acoustique *operando* pour un réacteur de broyage fermé, ainsi que les verrous ayant guidé les choix de conception, notamment les volumes des signaux traités et l’absence de vérité terrain sur les phénomènes internes au réacteur. Le chapitre 3 détaille la méthodologie retenue pour répondre à ces contraintes : un développement itératif au rythme hebdomadaire, rythmé par des retours réguliers de l’équipe de l’ICGM, et une architecture logicielle centrée sur `session_state` permettant de faire persister l’information d’une page à l’autre malgré le modèle d’exécution particulier de Streamlit. Le chapitre 4 présente les développements techniques associés : prétraitement du signal (sous-échantillonnage, filtrage direct ou IFIR selon la finesse de la bande recherchée), indicateurs statistiques par fenêtre glissante, méthode énergétique par bandes harmoniques, données complémentaires de température et export traçable des résultats. Le chapitre 5 illustre l’ensemble de cette pipeline sur un signal réel, puis présente un cas d’usage mené en autonomie par un utilisateur final de l’ICGM, avant de revenir sur les retours d’usage recueillis tout au long du stage.

L’application produite répond aux objectifs fixés en introduction : les méthodes héritées des stages précédents sont reprises, vérifiées et consolidées, de nouvelles briques sont intégrées lorsque cela s’avère pertinent (filtrage IFIR notamment), et l’ensemble est rassemblé au sein d’une pipeline reproductible, documentée et livrée à l’équipe de l’ICGM sous la forme d’une version stable (V7). Les retours recueillis, tant sur les corrections d’anomalies que sur les suggestions ergonomiques intégrées, ainsi que l’exemple d’utilisation autonome présenté au chapitre 5, confirment que l’outil est approprié par ses utilisateurs finaux au-delà d’un simple usage de démonstration.

Sur le plan personnel, ce stage permet une montée en autonomie sur le développement d’une application web en Python, depuis les choix d’architecture jusqu’à la prise en compte de retours d’usage, ainsi qu’un travail de compréhension et de vérification d’algorithmes de traitement du signal non triviaux à implémenter, en particulier autour du filtrage IFIR et du formalisme des indicateurs statistiques par fenêtre glissante. La conduite du projet en autonomie, avec un contact aux utilisateurs finaux limité aux échanges hebdomadaires et

au déplacement initial à Montpellier, développe également une communication technique, pour le tuteur de stage et les futurs développeurs de l'application, et non technique, pour une équipe de chimistes non forcément familière des algorithmes employés (cf. section 3.2, documentation). Cet exercice de vulgarisation ne se limite pas à la documentation écrite : il structure des choix faits au sein de l'application, à travers les infobulles associées aux paramètres de traitement du signal (cf. section 5, retours d'usage) et le `README`, rédigé en langage accessible pour expliquer le principe des méthodes employées (downsampling, filtrage, indicateurs, spectrogramme) sans recourir au formalisme mathématique détaillé dans ce rapport. Traduire un même concept technique à deux niveaux de profondeur différents, celui de ce rapport et celui destiné à l'utilisateur final, constitue un exercice complet, distinct de la seule implémentation des méthodes. L'usage encadré de l'IA générative, discuté en section 6.2.1, constitue par ailleurs un appui pour la correction de certains bugs et la factorisation du code, sans jamais se substituer à la maîtrise du code produit.

Plusieurs perspectives se dégagent de ce travail. À court terme, l'intégration du signal Raman à l'application, actuellement traité hors pipeline par les utilisateurs souhaitant confronter leurs résultats acoustiques à une mesure chimique de référence (cf. section 5.3), constituerait un prolongement envisageable de la page de données complémentaires. Le dispositif expérimental disposant de deux microphones, l'un placé au niveau du broyeur et l'autre au niveau du réacteur, une piste complémentaire consisterait à exploiter conjointement ces deux enregistrements par des méthodes de filtrage adaptatif (filtrage de Wiener ou de Kalman), dans l'optique de séparer la contribution propre au bruit mécanique du broyeur de celle associée aux phénomènes se déroulant au sein du réacteur, et d'en extraire des caractéristiques (*features*) acoustiques plus spécifiquement corrélées à l'évolution de la réaction chimique. Le déploiement sur Streamlit Community Cloud, engagé en fin de stage mais dont la mise en ligne effective reste à finaliser avec l'équipe de l'ICGM, permettrait par ailleurs de faire connaître l'application à la communauté au-delà d'un usage strictement local.

Enfin, ce stage permet de développer des compétences dépassant le seul cadre du traitement du signal : conduite de projet en autonomie, structuration d'une documentation à destination de publics différents, gestion des priorités entre nouvelles fonctionnalités et corrections d'anomalies, et dialogue avec des utilisateurs attentifs. Ces compétences, associées à la maîtrise acquise sur le développement d'applications Streamlit, s'inscrivent dans la continuité de la formation d'ingénieur en signal, image, communication et multimédia (SICOM) et trouvent un prolongement direct dans les missions d'ingénieur en traitement du signal.

Références bibliographiques

- [1] César LEROY et al. « Operando acoustic analysis : a valuable method for investigating reaction mechanisms in mechanochemistry ». In : *Chemical Science* 13.21 (2022), p. 6328-6334. ISSN : 2041-6520, 2041-6539. DOI : 10.1039/D2SC01496C. URL : <https://pubs.rsc.org/sc/article/13/21/6328-6334/786061>.
- [2] César LEROY et al. « Listening to the Formation of Polymorphs in a Ball Mill ». In : 97.5 (2025), p. 2922-2931. DOI : 10.1021/acs.analchem.4c05608. URL : <https://pubs.acs.org/doi/10.1021/acs.analchem.4c05608>.
- [3] O HERRMANN, L. R RABINER et D. S. K CHAN. *Practical Design Rules for Optimum Finite Impulse Response Low-Pass Digital Filters*. 1973. URL : <https://dn760108.eu.archive.org/0/items/bstj52-6-769/bstj52-6-769.pdf>.
- [4] *SciPy oaconvolve documentation*. URL : <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.oaconvolve.html>.
- [5] Jérôme ANTONI et al. « : Formulation d'indicateurs optimaux pour la détection de défauts ». In : 2023, p. 621-624. URL : https://gretsi.fr/data/colloque/pdf/2023_antoni1225.pdf.
- [6] *SciPy ShortTimeFFT documentation*. URL : <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.ShortTimeFFT.html>.
- [7] Jérôme ANTONI et al. « Optimality of the mean-square value, skewness, kurtosis, and synentropy for detection of weak additive fault signatures in (nearly) Gaussian noise ». In : *Mechanical Systems and Signal Processing* 245 (fév. 2026), p. 113828. ISSN : 08883270. DOI : 10.1016/j.ymssp.2025.113828. URL : <https://linkinghub.elsevier.com/retrieve/pii/S0888327025015298> (visité le 18/08/2026).
- [8] Jérôme ANTONI et al. « On the design of Optimal Health Indicators for early fault detection and their statistical thresholds ». In : *Mechanical Systems and Signal Processing* 218 (sept. 2024), p. 111518. ISSN : 08883270. DOI : 10.1016/j.ymssp.2024.111518. URL : <https://linkinghub.elsevier.com/retrieve/pii/S0888327024004163> (visité le 18/08/2026).
- [9] Jérôme ANTONI et Pietro BORGHESANI. « A statistical methodology for the design of condition indicators ». In : *Mechanical Systems and Signal Processing* 114 (jan. 2019), p. 290-327. ISSN : 08883270. DOI : 10.1016/j.ymssp.2018.05.012. URL : <https://linkinghub.elsevier.com/retrieve/pii/S0888327018302632> (visité le 18/08/2026).

-
- [10] I SELESNICK. *The Remez Exchange Algorithm for FIR Filter Design*. URL : <https://eeweb.engineering.nyu.edu/iselesni/EL713/remez/remez.pdf>.
- [11] *Matlab IFIR documentation*. URL : <https://fr.mathworks.com/help/dsp/ref/ifir.html>.

A. Annexes

Bilan de compétences et prévisionnel 3A

Ce bilan propose un positionnement global vis-à-vis du référentiel de compétences Ense³ à l'issue de mon stage de M1, ainsi que les pistes envisagées pour atteindre le niveau 3 de chaque compétence avant la fin de mon PFE.

Le stage réalisé au GIPSA-lab constitue le socle principal de ce positionnement. Il m'a permis de valider pleinement la compétence **C2 - Concevoir une solution face à un problème technique** : la conception de l'application de surveillance acoustique, développée en autonomie sur l'ensemble du stage, représente largement les 30 heures minimales attendues et fait l'objet d'un écrit réflexif détaillé en annexe C2.2. Ce même stage me permet également de considérer comme partiellement acquises deux composantes de la **C4 - Évoluer dans un environnement complexe** : l'animation d'un projet, à travers le pilotage en autonomie du cycle hebdomadaire de livraison (versions V0 à V7) et l'arbitrage constant entre nouvelles fonctionnalités et corrections remontées par les utilisateurs ; et le regard critique sur les règles professionnelles, via mon REX 1A déjà validé. Restent à consolider l'aspect « informer, expliquer, argumenter », que je considère en bonne voie grâce aux soutenances et rapports déjà produits mais que je souhaite confirmer lors de mon PFE, ainsi que le volet interculturel, pour lequel je n'ai pour l'instant aucune mobilité réalisée. Ce dernier point sera directement complété par mon semestre Erasmus à l'Università degli Studi di Trento, qui me permettra à la fois de comptabiliser la durée de mobilité attendue et de vivre une véritable immersion dans un environnement d'enseignement international.

La compétence **C6 - Agir en professionnel responsable et en acteur d'une transition durable** est celle sur laquelle mon positionnement est le plus avancé, avec toutefois des zones encore à traiter. Le traitement de la documentation comme un livrable à part entière durant le stage : README utilisateur, document développeur INFOS.md, dépôt GitHub structuré, illustre concrètement ma capacité à transmettre un travail compréhensible et pérenne. Les 12 heures de cours que j'ai dispensées à des élèves de 1A en traitement du signal, langage C et calcul scientifique, tracées par une attestation signée, valident de la même manière le volet auto-formation/formation des autres. Le Sulitest est validé et le rapport d'analyse d'impacts socio-environnementaux de ma structure d'accueil est livré en annexe du présent rapport, ce qui couvre une bonne partie du volet évaluation

d'impacts : il ne me reste qu'à finaliser mon bilan carbone personnel portant sur mes mobilités pour compléter ce dossier de traces. Deux composantes restent en revanche à construire : l'analyse réflexive d'une erreur ou d'un échec ; et l'engagement dans la société, pour lequel je dispose d'une piste concrète : accompagner les jeunes joueurs (8-16 ans) du club de football dans lequel j'ai évolué pendant quinze ans, à travers de l'aide aux devoirs et un suivi lors de stages vacances mais qui reste à valider avec le responsable compétences de l'école. Cette proposition m'a directement été faite par le président du club de foot qui cumule l'obtention de labels qualité jeunes depuis quelques années et qui cherche à s'imposer comme une école de foot leader en Auvergne-Rhône-Alpes.

Les compétences **C1 - Modéliser des phénomènes naturels et physiques** et **C5 - Prendre des initiatives, innover et entreprendre, apprendre par la recherche** sont celles pour lesquelles mon positionnement actuel est le plus limité. Je n'ai pour l'instant aucune expérience de modélisation aboutie représentant les 30 heures attendues ; je compte sur mon PFE ainsi que sur les enseignements d'ingénierie biomédicale suivis en Erasmus (*Ultrasound technologies for medical applications, Robotics for biomedical engineering*) pour développer cette compétence dans un cadre concret et cohérent avec mon projet professionnel. Pour la C5, le travail de reconstruction de la méthode de filtrage IFIR à partir de la littérature technique, en l'absence d'implémentation Python disponible, constitue une première amorce de démarche proche de la recherche appliquée, mais reste largement insuffisant pour couvrir seul le seuil de 60 heures requis. Je souhaite consolider cette compétence lors de mon PFE si celui-ci comporte une composante recherche, ou à défaut via un projet dédié en 3A, en veillant à mettre en place la traçabilité nécessaire dès le lancement de l'activité.

En synthèse, mon stage de M1 me permet d'ores et déjà de couvrir solidement les compétences C2 et une large part de C4 et C6, tout en révélant clairement les axes de travail prioritaires pour la suite de mon cursus : la mobilité Erasmus à Trento pour le volet interculturel et pour une première approche de la modélisation en contexte biomédical par exemple, une activité de recherche dédiée pour valider pleinement C5, et deux démarches personnelles simples mais à engager sans tarder pour compléter C6 : l'analyse d'une erreur vécue et un engagement associatif concret.

Compétence C2 - Concevoir une solution face à un problème technique

Durant mon stage de 11 semaines au GIPSA-lab, en collaboration avec l'Institut Charles Gerhardt Montpellier (ICGM), j'ai eu pour mission de transformer un ensemble de scripts Python hétérogènes, issus de stages précédents sur le suivi acoustique de synthèses mécano-chimiques, en une application unique, robuste et directement utilisable par une équipe de chimistes. Ce projet m'a confronté pour la première fois à la conception complète d'un produit logiciel, de la compréhension des besoins jusqu'à sa livraison à des utilisateurs réels, en cohérence avec mon projet professionnel d'ingénieur en traitement du signal.

Le besoin initial n'était pas formalisé : je suis parti d'un problème d'usage diffus (des méthodes de traitement du signal déjà validées, d'autres non, aucun cahier des charges préexistant. Je l'ai construit moi-même, progressivement, lors d'un déplacement à Montpellier pour rencontrer l'équipe et découvrir le dispositif expérimental, puis affiné au fil d'un cycle hebdomadaire de retours utilisateurs sur plusieurs versions successives (V0 à V7).

La conception a dû concilier plusieurs types de contraintes. Des contraintes techniques d'abord : volumes des signaux acoustiques (une heure d'enregistrement à 44,1 kHz), coût de calcul du filtrage sur des bandes de fréquence étroites, résolu par la conception de filtres IFIR absents des bibliothèques Python et reconstruits à partir de la littérature. Des contraintes de maintenabilité ensuite : architecture centrée sur `session_state` et registres centralisés, pensée pour limiter le coût des évolutions futures du code par l'équipe du laboratoire. Enfin, deux contraintes de nature sociétale et environnementale structurent le projet dans son ensemble. D'une part, la finalité même de l'outil s'inscrit dans une démarche de développement durable : la mécano-chimie, dont ce projet permet un suivi plus fin, constitue une alternative de synthèse dite « verte », qui réduit le recours aux solvants par rapport aux voies de synthèse classiques en solution ; en facilitant l'appropriation de ces méthodes de suivi par les chimistes, l'application contribue indirectement à la diffusion de cette approche moins polluante de la synthèse chimique. D'autre part, une contrainte d'accessibilité a orienté l'ensemble des choix de conception : l'outil devait être utilisable par un public non formé à la programmation, ce qui a directement motivé le choix technologique de Streamlit ainsi que le traitement de la documentation comme un livrable à part entière (README en langage accessible, infobulles sur les méthodes de traitement du signal), plutôt que comme une étape annexe du développement.

La solution retenue n'est pas l'adaptation d'un outil existant mais une architecture complète, conçue et dimensionnée pour ce cas d'usage précis : pipeline organisée en six pages Streamlit, façade unique vers les fonctions de traitement, dossier de modules organisé

par thématique, et mécanisme de filtrage IFIR reconstruit à partir de la documentation Matlab existante et d'articles de référence, faute d'implémentation Python disponible.

La solution n'a pas été testée dans un cadre contrôlé mais directement par ses utilisateurs finaux, sur leurs propres données, en conditions réelles d'utilisation : cycle hebdomadaire de livraison et de retours sur neuf semaines, cas d'usage mené en autonomie complète par un stagiaire de l'ICGM sur une synthèse réelle, et correction de défauts de fonctionnement identifiés par les équipes en continu.

L'étape la plus difficile a été d'arbitrer chaque semaine entre l'avancement de nouvelles fonctionnalités et la correction des défauts remontés, avec un contact aux utilisateurs limité à des échanges par mail. Avec plus de recul, j'aurais formalisé plus tôt un système de suivi structuré des retours, plutôt que de les traiter au fil de l'eau. Ce stage m'a transformé dans ma capacité à concevoir un produit destiné au public, en conciliant rigueur de conception et communication accessible, tout en gardant présentes des contraintes qui dépassent le seul cadre technique : utilité environnementale de l'outil et accessibilité de son usage. C'est une compétence que je pourrai valoriser en entretien de recrutement, en montrant comment un besoin peu défini au départ a été transformé, par itérations argumentées et sous contraintes multiples, en un outil livré et adopté par ses utilisateurs.

Cette attestation accompagne la procédure de validation de la compétence à concevoir une solution face à un problème technique. Ce document doit être complété pour chaque expérience de conception en situation d'autonomie seul ou en binôme. Remarque : la situation d'autonomie peut-être obtenue dans des groupes projet de grande taille dès l'instant où le travail a été séparé clairement entre les membres de l'équipe et dès lors que l'encadrant du travail sait à qui ont été attribuées les différentes tâches.

NOM et Prénom	SANTALUCIA Morgan	
Année de formation et Filière	Année universitaire 2025/2026 -- SICOM 2A	
Promotion (année de diplomation prévue à votre arrivée à l'école)	2027	
Activité concernée (cocher la case correspondante)	Bureau d'études prévu dans la maquette	
	Projet de 1 ^{ère} année (projet personnel)	
	Projet de 2 ^{ème} année (projet ingénierie, industriel, recherche)	
	Projet de 3 ^{ème} année	
	Stage d'assistant ingénieur	✓
	PFE	
	Mission en entreprise (pour les alternants)	
	Autre, préciser :	
Nom des étudiants éventuels avec qui le travail a été réalisé	• • •	

Nom du responsable d'activité qui pourra attester de votre activité :

Nom, Prénom : GRANJON Pierre

Qualité : Tuteur de stage laboratoire -- Enseignant - chercheur à Grenoble INP

Pour les stages et missions en entreprises, nom de l'enseignant(e) qui pourra co-attester de cette activité :

Nom, Prénom : MARS Jerome

Qualité : Tuteur de stage école -- Enseignant - chercheur à Grenoble INP

Description brève du contexte (problématique, enjeux, objectifs) et de l'activité (démarche, principaux résultats, outils mis en œuvre)	Suivi de synthèses mécano-chimiques par analyse acoustique. L'objectif était de concevoir une application unique et robuste, directement exploitable par une équipe de mécano-chimistes. Le travail comprenait l'analyse des besoins utilisateurs, la conception progressive du cahier des charges, le développement itératif en Python/Streamlit. La livraison comprenait une application complète (~4000 lignes de code, pipeline de prétraitement, indicateurs statistiques, analyse énergétique, export traçable), adoptée en usage quotidien par l'équipe de l'ICGM.		
Auto-positionnement du niveau de développement de chaque composante essentielle à l'issue de l'expérience	NIVEAU 1 Le cahier des charges est compris Une contrainte technique est prise en compte Une solution existante est identifiée La solution est évaluée en environnement connu et maîtrisé pour un fonctionnement nominal	NIVEAU 2 Le cahier des charges est modifié à juste titre Plusieurs contraintes de même nature sont prises en compte Une solution existante est adaptée à juste titre La solution est évaluée en environnement connu et maîtrisé pour différentes situations de fonctionnement	NIVEAU 3 Le cahier des charges est élaboré Plusieurs contraintes de différentes natures (dont DDRS) sont prises en compte Une solution est élaborée et dimensionnée La solution est évaluée en environnement réel de fonctionnement
Argumenter cet auto-positionnement et expliciter la combinaison des ressources mobilisées (savoirs, savoir-faire, savoir-être)	Le cahier des charges est élaboré de toutes pièces avec les utilisateurs, en prenant en compte des contraintes multiples (techniques, maintenabilité, DDRS avec la mécano-chimie « verte », accessibilité). La solution est dimensionnée spécifiquement, avec une architecture complète et des filtres IFIR reconstruits en réponse directe aux besoins utilisateurs. L'évaluation se fait en environnement réel, par les utilisateurs finaux, sur 9 semaines. Sont mobilisés des savoirs en traitement du signal, Python et recherche bibliographique, un savoir-faire en développement logiciel, ainsi que de l'autonomie et des capacités de communication technique et non technique.		
Analyser les principales difficultés et les facilités identifiées durant l'expérience	La conception logicielle sur Streamlit est très facile d'accès et rapide à prendre en main. L'affranchissement de la logique frontend/backend rend le travail plus agréable pour un stagiaire en traitement du signal dont le développement d'applications n'est pas l'activité première. Les choix d'architecture sont par ailleurs bien plus difficiles pour un profil de ce type. Le cahier des charges étant large, le partitionnement des codes et l'ordre d'implémentation sont deux problématiques particulièrement délicates à aborder		
Et si c'était à refaire... préciser des pistes d'amélioration personnelle si cette expérience se reproduisait	Lors de futures expériences de développement d'application, un travail d'architecte plus conséquent sera exécuté. En effet, une fois les méthodes de traitement validées, c'est leur ordre d'implémentation et leur positionnement dans l'architecture du code qui rendent le développement plus ou moins efficace. Une prise en considération encore plus approfondie des besoins utilisateurs permettrait de gagner du temps au niveau de la programmation grâce à un prototype plus élaboré		
Période de l'activité (date début et fin) et temps estimé consacré à l'activité ¹	Du 18/05/2026 au 31/07/2026 à raison de 5 jours de 7h de travail hebdomadaires => ~350h au total		

Date et signature de l'étudiant(e)
01/08/28 

¹ Exprimé en heures totales, en h/semaine, en jours, en semaines, ...

Partie administrative à remplir par le responsable d'activité et l'enseignant évaluateur le cas échéant (stages et missions en entreprises uniquement).

L'école demande à l'ensemble des élèves-ingénieurs de justifier à la fin de leur cursus d'un minimum de 30h de mise en application des connaissances et savoir-faire de conception. La reconnaissance et la traçabilité de ces expériences sont donc primordiales pour la diplomation de nos élèves. L'école vous remercie pour le temps et le soin que vous voudrez bien porter à la lecture de ce document complété par l'élève à l'issue d'une telle expérience de modélisation, et à son évaluation ci-dessous.

Groupe de travail : ce travail de conception a été réalisé²

seul	en binôme
------	-----------

Quantité de travail : combien d'heures totales (encadrées et non encadrées) de travail de l'élève pouvez-vous attester sur cette activité de conception ?

moins de 10h	entre 10 et 30h	entre 30h et 60h	plus de 60h	non évaluable ³
--------------	-----------------	------------------	-------------	----------------------------

Autonomie : combien de fois avez-vous rencontré l'élève lors de l'activité de conception ?

moins de 3 fois	de 3 à 6 fois	plus de 6 fois
-----------------	---------------	----------------

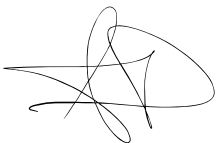
Estimation du niveau de développement de chaque composante essentielle pour le travail réalisé :

NIVEAU 1	NIVEAU 2	NIVEAU 3
Le cahier des charges est compris	Le cahier des charges est modifié à juste titre	Le cahier des charges est élaboré
Une contrainte technique est prise en compte	Plusieurs contraintes de même nature sont prises en compte	Plusieurs contraintes de différentes natures (dont DDRS) sont prises en compte
Une solution existante est identifiée	Une solution existante est adaptée à juste titre	Une solution est élaborée et dimensionnée
La solution est évaluée en environnement connu et maîtrisé pour un fonctionnement nominal	La solution est évaluée en environnement connu et maîtrisé pour différentes situations de fonctionnement	La solution est évaluée en environnement réel de fonctionnement

Remarques éventuelles :

.....

.....

Date et signature du responsable d'activité	Date et signature de l'enseignant évaluateur (si stage ou mission en entreprise)
24/08/2026 Pierre Granjon 	

² Cette activité ne sera pas comptabilisée pour des groupes de plus de 3 élèves.

³ Si non évaluable, en préciser les raisons dans la rubrique Remarques éventuelles.

Analyse DDRS

Afin d'analyser la prise en compte des enjeux socio-environnementaux au GIPSA-lab, j'ai réalisé un entretien avec Sylvain Gerber, statisticien au laboratoire depuis 13 ans et membre du groupe de travail consacré à la responsabilité sociétale et environnementale de la recherche. Cet échange m'a permis de compléter les informations disponibles sur l'intranet et de mettre ces éléments en parallèle de ma propre mission de stage.

Le GIPSA-lab a mis en place plusieurs structures dédiées à ces enjeux. La Commission Ecolo-gipsa, créée en 2018, rassemble plusieurs membres du laboratoire autour de la réduction de son impact environnemental. Ses premières actions ont notamment porté sur la réalisation d'un bilan des émissions de gaz à effet de serre (GES) en 2019 avec le calculateur Labo1.5 du CNRS, en prenant en compte les déplacements domicile-travail et professionnels, les consommations des bâtiments et les achats de matériel. Une limite identifiée lors de l'entretien concerne cependant la prise en compte des serveurs et infrastructures informatiques, qui peuvent représenter un poste important pour un laboratoire de recherche comme le GIPSA-lab.

La commission dispose aujourd'hui d'un budget d'environ 5000 euros par an et mène à la fois des actions de sensibilisation et des actions concrètes. Des Fresques du climat et des journées de sensibilisation sont par exemple proposées aux nouveaux arrivants, tandis que des livres et affichages permettent de maintenir cette sensibilisation au quotidien. Une charte écologique a également été construite en 2021, après plusieurs réunions de travail, puis intégrée au règlement intérieur du laboratoire. D'autres initiatives concernent directement le fonctionnement du site : installation de composteurs, ateliers de réparation de vélos et de matériel informatique, homogénéisation des poubelles de tri et installation de thermostats. Selon les informations recueillies lors de l'entretien, le passage du chauffage au gaz au chauffage urbain aurait par ailleurs permis de diviser environ par deux les émissions de CO₂ liées au chauffage.

La démarche concerne également les pratiques quotidiennes du laboratoire. Une attention est portée aux prestataires et aux achats lors de l'organisation d'événements, avec notamment une préférence pour les produits locaux, les options végétariennes et la vaisselle réutilisable. Le laboratoire semble par ailleurs bénéficier d'une bonne sensibilisation de son personnel à ces problématiques, avec un intérêt qui a globalement progressé depuis la création de la commission. D'après Sylvain Gerber, le GIPSA-lab est même régulièrement sollicité par d'autres laboratoires souhaitant s'inspirer de ses initiatives.

La dimension sociale est également prise en compte à travers le Comité Égalité-Parité, créé en 2021, qui intervient sur les questions de discrimination, de sexisme, de racisme, de handicap ou encore de harcèlement. Enfin, le groupe GIPSA-RSER constitue un espace

de discussion autour de l'éthique environnementale et sociétale de la recherche. Cette organisation montre que les enjeux DDRS ne sont donc pas uniquement traités comme une question de réduction des émissions, mais également comme une réflexion sur le fonctionnement du laboratoire et ses pratiques de recherche.

Cette démarche me semble relativement structurée, mais son suivi pourrait encore être amélioré. Le bilan GES constitue une première base, mais je n'ai pas identifié de suivi régulier regroupant l'ensemble des indicateurs et certaines consommations restent difficiles à évaluer, notamment celles liées aux infrastructures informatiques. Un suivi plus précis de ces postes permettrait d'identifier plus facilement les principaux leviers d'action.

Ma mission de stage présente un lien avec ces enjeux, mais de manière indirecte. J'ai développé une application Streamlit permettant à des chimistes d'analyser plus facilement des signaux acoustiques issus d'expériences de mécanochimie. Cette approche de synthèse peut constituer une alternative moins consommatrice de solvants à certaines voies classiques. Mon travail ne permet toutefois pas de mesurer directement cet impact : il facilite surtout l'utilisation d'un outil de suivi de ces réactions et peut ainsi contribuer indirectement à leur étude.

Les choix techniques réalisés pendant le stage peuvent également être analysés sous l'angle de la sobriété numérique. Le sous-échantillonnage des signaux, l'optimisation des calculs et la mise en cache permettent de limiter les traitements inutiles. La maintenabilité de l'application et sa documentation peuvent également éviter de reproduire du travail ou de reconstruire l'outil à chaque évolution. Ces choix répondaient d'abord à des contraintes techniques, mais ils ont également une dimension environnementale, bien qu'elle n'en soit pas la motivation première.

Avec le recul, je n'avais cependant pas intégré explicitement l'éco-conception au cahier des charges. J'aurais pu davantage mesurer le coût informatique des différentes méthodes, réfléchir au stockage des données et comparer les conséquences d'un déploiement local ou dans le cloud. Il faut également considérer un possible effet rebond : rendre l'analyse plus simple peut entraîner une augmentation du nombre de calculs réalisés et donc des ressources consommées, comme nous avons pu le voir en cours de Numérique Responsable cette année en SICOM.

Cet entretien m'a finalement permis de comprendre que la responsabilité de l'ingénieur ne se limite pas à produire une solution techniquement efficace. Les choix de conception, de calcul, de stockage ou de déploiement ont eux aussi des conséquences. Pour mes prochains projets, je souhaite donc intégrer ces questions plus tôt dans la définition du besoin, plutôt que de les considérer uniquement une fois la solution technique construite.